

COMP4423 – Assignment 1

Triangle-Brick Image Generation

Huang Haoran (24101322D)

Repository: https://github.com/Retr0-rgb-lab/COMP4423_Assignment1

1 Introduction

Given a single image, the system converts it into a mosaic of triangular bricks that visually approximates the original, under a hard budget of at most 10,000 individual right-isosceles triangles, with no gaps or overlaps, where each brick carries one uniform face colour determined from the image region it covers. Multiple brick sizes and multiple colours are allowed.

The no-gaps/no-overlaps rule is a hard constraint, so I did not want to rest it on the construction argument alone. `code/verify_coverage.py` checks it directly for all three delivered tessellations: cells are distinct and grid-aligned, each cell's two triangles satisfy the exact area identity $\text{area}(a) + \text{area}(b) = s^2$ (shoelace on the vertices), and rasterising the triangles leaves **0 uncovered pixels** in the delivered crop. The test is geometric rather than a rasterised multiplicity count because `cv2.fillPoly` paints a one-pixel fringe along every internal diagonal — a rasterisation convention, not an area overlap, and one I quantify separately in Section 6.4. All three pass.

Because the budget is fixed, the real work is deciding where the detail should go. More triangles capture more edges and texture, but they also make each brick's colour estimate noisier (a tiny triangle averages very few pixels, so sensor noise starts to look like detail), and none of that addresses the colour error introduced by mapping each region to a fixed palette entry. I treat the three as separate things and let each task attack whichever matters most at that point.

I built the system in four stages, and each stage surfaced a problem that shaped the next. Task 2 fixes the tessellation to equal-size bricks with three colours; measuring its output showed that a uniform grid does not align with image content, which became the motivation for Task 3. Task 3 makes both the geometry and the palette adaptive, which is where the budget-allocation question above is actually answered. Task 4 runs the same pipeline in a live camera loop, where the speed and stability problems the offline setting hides become the main engineering work.

Test image. The test photo for this report is `code/pics/sky.jpg` (1706×1279), a canal-side cityscape — brick wall, trees, distant buildings — captured on the camera used for Task 1. Its dominant signal is luminance variation rather than hue variation, which has a measurable effect on the colour-quantisation experiments (Sections 4 and 5).



Figure 1. The test photo `code/pics/sky.jpg` (1706×1279): a canal-side cityscape with brick architecture, trees and distant buildings.

2 Task 1 — capture and display

Task 1 is to call the camera, capture a frame, and display it. The program is `code/capture.py`: it opens the default camera with `cv2.VideoCapture(0)` [9], reads one frame, prints its shape and channel order, and shows it with `cv2.imshow` until a key is pressed, at which point the capture is released and the window closed. Running it on the machine with the camera attached:

```
python code\capture.py
```

That is the Task 1 deliverable in full. The script also accepts `-from-file PATH` to read a still frame instead of opening a camera, and `-save PATH` to write the frame to disk, both of which let the capture path be exercised and inspected without a live camera in front of it. The frame shown in Fig. 1 is the test photo this report works from.

3 Method — the shared toolkit

The four tasks share one geometry, colour and rendering core (`code/brick_geom.py`, `brick_color.py`, `brick_render.py`, `brick_metrics.py`). I kept the decisions that every task depends on in this one place, so later sections only have to add the one thing each task changes. Table 1 summarises them.

This section and the ones after it answer, in order, the six method questions the report is organised around. Where each one is answered:

question	answered in
Q1 design and test	this section; Sections 4, 5 and 6 per task
Q2 robustness in a real scenario	Section 6.6
Q3 problems and how I solved them	Sections 4.3, 5.5 and 6.7
Q4 how I used GenAI	Section 7
Q5 how GenAI understood the tasks	Section 7.1
Q6 GenAI limitations and improvement	Section 8

How I designed and tested it (Q1). The design puts every choice that all four tasks share into one geometry/colour/rendering core (Table 1) so that each task is a thin policy layer on top of it. Testing is verification-driven rather than example-driven: two headless drivers assert properties instead of eyeballing outputs. `code/verify_coverage.py` checks the no-gaps/no-overlaps rule geometrically for all three delivered tessellations (cells distinct and grid-aligned; each cell's two triangles satisfy $\text{area}(a) + \text{area}(b) = s^2$; zero uncovered pixels in the rasterised crop), and `code/task4_verify.py` is an assertion suite (partition equivalence, triangle generation, render, palette cache, end-to-end, row-run means, temporal stability) that must pass before any performance number is believed. Qualitative checks are then layered on top as figures and tables, and every reported number is regenerated from the stored `*_metrics.json` under a fixed seed rather than transcribed. The per-task design choices and their measured consequences are in Sections 4, 5 and 6.

Component	Choice	Rationale
Brick geometry	Square cell cut by one diagonal , direction alternates by checkerboard parity	Guarantees right-isosceles, gap-free, overlap-free tiling; alternation avoids a visible single-directional smear
Brick size	Length of the equal legs, in base-grid units; Task 2 fixed, Task 3 power-of-two sizes $\{1, 2, 4, 8, 16, 32, \dots\}$	A brick’s size is its equal-leg length; powers of two make quadtree halving exact
Colour per brick	Mean BGR of the pixels under the triangle’s mask (<code>cv2.fillPoly + cv2.mean</code>)	Robust to sub-region detail; the brick’s colour is read from the region it covers
Palette (Task 2)	Multi-Otsu [5] on BT.601 luminance (primary), K-Means [6] $K = 3$ and a fixed 33%/67% percentile split as baselines	Otsu thresholds adapt to the scene histogram; baselines make the results section a real comparison
Palette (Task 3)	K-Means on CIE-Lab ($K = 8/16$), Median-Cut [8] as baseline	Lab is perceptually uniform; median cut is cheap and deterministic
Tessellation (Task 3)	Top-down RDO quadtree , split priority ΔMSE per triangle ($\Delta\text{SSE}/6$); region-merge as dual algorithm	Rate-distortion allocation: equal per-triangle units make different sizes comparable
Border	1-px #3C3C3C polyline per triangle	Visual separation; boundaries are drawn, not counted as brick colours
Metrics	10-item metric suite (9 quality + 1 constraint): PSNR, SSIM [1], MS-SSIM (4-level mean, a practical variant of the multi-scale SSIM of Wang et al. [2]), $\Delta\text{E}2000$ (CIEDE2000 [3]), Edge F1/Precision/Recall (Canny [4] + 3-px tolerance), edge-alignment correlation (EAC; Pearson correlation of Sobel responses — labelled “EPI” below for short) , Quantisation Error, Budget Utilisation	Each metric catches a different failure mode; the EAC/EPI metric specifically measures whether triangle boundaries align with real edges

Table 1. Design decisions of the shared geometry/colour/rendering toolkit.

Budget semantics. The limit is 10,000 *triangles* (“bricks”); a quadtree leaf cell produces 2 triangles, so a partition of 4,995 cells exhausts the budget at 9,990 triangles. All Task 3 runs use 9,988–9,990 triangles, which is 99.88–99.90% of the budget.

Metric evaluation region. All PSNR/SSIM/ ΔE /Edge/EPI numbers in Sections 4–5 are computed on the rendered canvas with the 1-px border drawn. The border is `#3C3C3C` and, on the chosen Task 3 render, covers 11.6% of pixels, so it is a constant confound shared by every run; its effect is quantified in Section 5.6. The 9 quality metrics are PSNR, SSIM, MS-SSIM, $\Delta E2000$, Edge F1/Precision/Recall, EPI and Quantisation Error, with Budget Utilisation tracked separately as a constraint. Appendix A gives the exact definition and the direction of each one, because several of them disagree with each other on purpose.

4 Task 2 — equal-size triangles, 3 colours

4.1 Design and testing

The tessellation is a uniform $S \times S$ grid cut by checkerboard diagonals. The grid step is the smallest S for which $2 \cdot M \cdot N \leq 10000$ ($M = \lceil H/S \rceil$, $N = \lceil W/S \rceil$). For the three colours I compare three quantisation methods on identical geometry and means. The primary one is Multi-Otsu [5]: an exhaustive two-threshold search over the BT.601 luminance histogram, with each class painted the mean BGR of its members. As baselines I also run K-Means [6] with $K = 3$ on BGR (K-Means++ [7] initialisation) and a fixed percentile split at 33%/67% of the luminance histogram, the last being a deliberately naive baseline that ignores scene statistics entirely.

How it was tested. All three share one geometry (59×78, $S = 22$, 9,204 triangles on `sky.jpg`) so only the quantiser differs; the full 10-item metric suite is computed per method. Outputs: `code/pics/task2/out_task2_compare.png` (original + 3 renders), `out_task2_residual.png` (per-pixel $\Delta E2000$ heatmaps), `out_task2_metrics_chart.png`.

4.2 Results

Metric	otsu	kmeans	fixed
PSNR up (dB)	16.15	16.16	15.58
SSIM up	0.332	0.334	0.286
MS-SSIM up	0.482	0.481	0.452
$\Delta E2000$ down	11.41	11.41	11.92
Edge F1 up	0.191	0.189	0.233
Edge Precision up	0.151	0.150	0.171
Edge Recall up	0.261	0.255	0.365
EPI up	-0.028	-0.031	-0.012
Quant Error down	8.47	8.48	8.98

Table 2. Task 2 results: three colour quantisers on identical geometry (59×78, $S = 22$, 9,204 triangles, `sky.jpg`).

Three things stand out in this table. First, otsu and kmeans are indistinguishable on this scene: the palettes differ by at most 2 per channel and SSIM by only 0.002 (0.332 vs 0.334). K-Means in 3-D BGR is collapsing onto the same 1-D luminance split that Multi-Otsu uses directly, because

the dominant signal here is luminance, not hue; its theoretical colour-resolution advantage never shows up on a near-grayscale image.

Second, and more interesting, **fixed** wins on every edge metric while losing on every colour and structure metric. Its uniform 33%/67% thresholds happen to place more of the original edges exactly on a colour boundary between adjacent triangles, so those edges survive as hard boundaries. Otsu and K-Means push their thresholds to perceptually meaningful splits (sky versus wall) and instead smooth over thin edges. There is a real trade-off here between perceptual fidelity and raw edge preservation, and which method is “better” depends on what one means by quality.

Third, EPI is negative for all three methods (about -0.03). A negative EPI means the output has weak edges where the original has strong ones and vice versa: the regular grid is introducing boundaries that have nothing to do with the image content. That is the quantitative signature of the uniform grid’s geometric mismatch, and it is the main reason I move to an adaptive tessellation in Task 3.

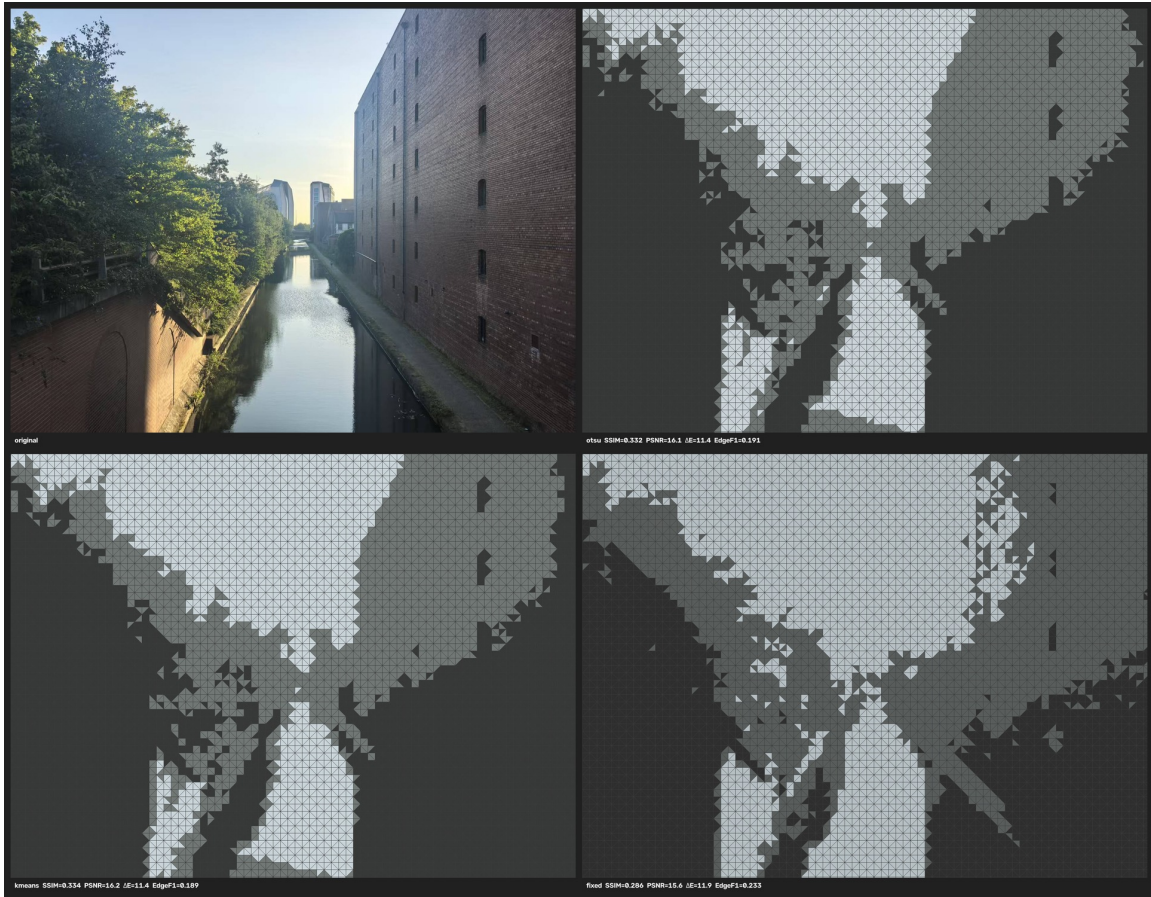


Figure 2. Task 2 comparison grid: the original image and the three colour-quantisation results (Multi-Otsu, K-Means, fixed percentile) on identical geometry, with their key metrics. **fixed** visibly collapses the sky into a mid-gray band (its 33%/67% split ignores scene statistics).

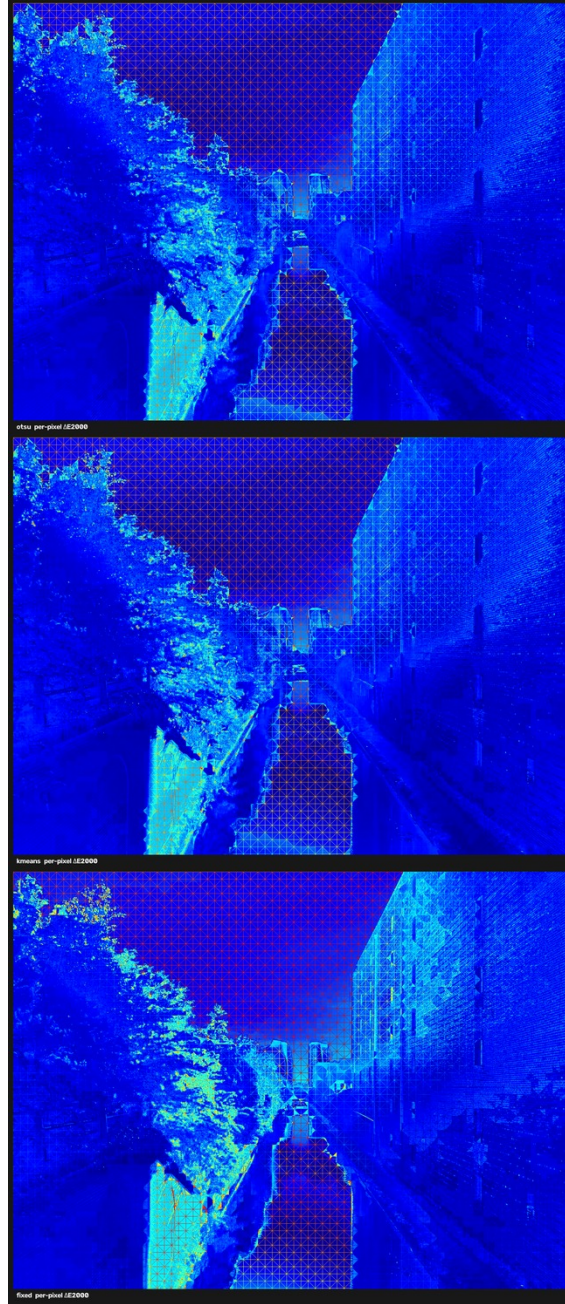


Figure 3. Per-pixel ΔE_{2000} residual heatmaps (JET, blue = small error) for the three quantisers. Otsu and K-Means are nearly identical with error concentrated in tree foliage; **fixed** adds a bright error band across the water.

Task 2 quantization-method comparison (sky.jpg)

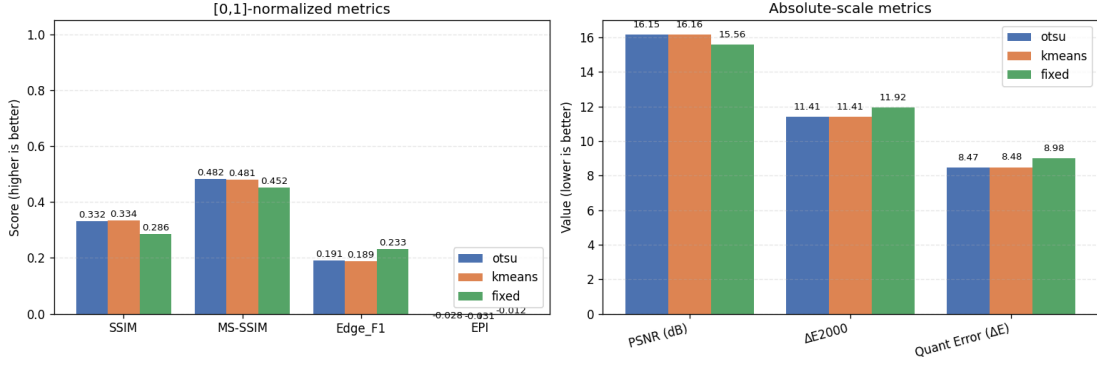


Figure 4. Task 2 metric summary across the three quantisers (normalised and absolute scales). Otsu $\approx$ K-Means on colour/structure; **fixed** wins only the edge family.

4.3 Problems found and solved

The first run produced only two triangles because `compute_grid` scanned S downward and returned the largest S that fit the budget, which for this image is a 1×1 grid. The code was syntactically valid but semantically wrong; the loop has to grow S upward and return the smallest S that stays under budget. I fixed it by inverting the scan (`code/brick_geom.py::compute_grid`).

The grid also left a black band. The floor-based grid dropped the trailing rows and columns that did not form a full $S \times S$ square (5 px on the right, 19 px on the bottom of `sky.jpg`), so those pixels stayed black and inflated the PSNR/SSIM error. I fixed this by switching to a `ceil` grid and reflect-padding the image to $M \cdot S \times N \cdot S$, rendering, then cropping back to the original size (`code/triangle_brick.py::preprocess`). After the fix SSIM rises from 0.3226 to 0.3316 and PSNR from 15.76 to 16.15 dB; the pre-fix numbers are preserved in git at commit `a5d2d76` if a direct before/after is needed.

The grid-selection bug came from an AI-written draft: the model produced the `compute_grid` loop with the scan direction inverted, the code was syntactically valid, and only a manual pass through the logic — tracing what the loop returns for a large image — showed it was wrong. I corrected it by hand. That episode is a reminder that AI-generated logic is not guaranteed to be correct, and that every such function needs its control flow checked step by step rather than trusted because it runs.

5 Task 3 — adaptive multi-size, multi-colour

5.1 Design and testing

I built on Task 2 by adding two adaptive pieces. The tessellation is now a **top-down RDO quadtree** that starts from a coarse grid and splits the highest-priority cell ($\Delta\text{MSE}/6$) while budget remains, and the palette is now **K-Means-Lab** with $K > 3$, with Median-Cut [8] kept as a baseline. I dropped the Task 2 primary quantiser (Multi-Otsu) because Otsu is a 1-D luminance method that cannot produce more than a small number of threshold classes. Task 3 needs an arbitrary K -colour palette, so I replaced it with a general clustering method on the perceptually uniform Lab space.

Region merging (dual algorithm). As a contrast to the top-down quadtree, I also implemented

a bottom-up region-merge (`brick_region_merge`). It starts from cells of the smallest allowed size and greedily merges an aligned 2×2 block of equal-size cells into one cell of twice the side while the triangle budget allows, using summed-area tables to score each merge candidate. The merge candidates are generated on the `target_size` grid so every merged block can itself form the next level's 2×2 block. On `sky.jpg` it terminates at 9,990 triangles with 44,415 merges and a final size distribution of only $\{16, 32\}$ (it merges all fine cells away), which is the property that drives its comparison against the quadtree in Section 5.2.

I designed a 15-run experiment grid that changes one variable at a time and holds everything else fixed, so that any difference in the metrics can be attributed to that one variable. Each group answers a specific design question:

group	changed variable (everything else fixed)	the design question it answers
A_ksweep	$K \in \{4, 8, 16\}$	Does a larger palette improve colour error, and at what cost to structure?
B_sweep	$S_{\min} \in \{2, 4, 8\}$ (..32)	Does dropping the smallest cells from the allowed set matter?
B2_smax	$S_{\max} \in \{32, 64, 128\}$	Does starting from a coarser grid (more budget headroom for deep splits) change the result?
C_algorithm	partition: quadtree vs region-merge	Which tessellation strategy — top-down RDO or bottom-up merging — is better?
D_palette	palette: K-Means-Lab vs Median-Cut	Which palette method, with geometry fixed, wins on colour and on structure?
E_priority	priority: Δ MSE vs Sobel edge-density	Does the split priority affect edge alignment, or is Δ MSE the right choice?

All runs use 9,988–9,990 triangles, i.e. 99.88–99.90% of the budget. The full table is at `code/pics/task3/summary/metrics_table.csv`.

5.2 Results (selected rows)

run	SSIM up	MS-SSIM up	Δ E2000 down	Edge F1 up	EPI up	Quant Err down
A:k04	0.384	0.513	11.08	0.330	0.064	9.25
A:k16	0.360	0.539	8.74	0.348	0.047	5.62
B2:smax_32	0.359	0.520	9.95	0.295	0.046	7.30
B2:smax_64	0.421	0.555	10.21	0.377	0.109	7.41
C:quadtree	0.359	0.520	9.95	0.295	0.046	7.30
C:region_merge	0.332	0.522	9.59	0.298	0.019	6.47
D:median_cut	0.375	0.526	10.61	0.322	0.052	7.76
E:prio_mse	0.370	0.518	10.22	0.320	0.050	7.33
E:prio_edgef1	0.374	0.499	9.83	0.213	-0.020	6.45

Table 3. Task 3 results (selected rows of the 15-run sweep; source `metrics_table.csv`).

(The rows `C:quadtree`, `B2:smax_32` and `D:kmeans_lab` are the same default run of the same configuration and reproduce the same values (SSIM 0.359, ΔE 9.95); `E:prio_mse` shares the configuration but differs slightly (SSIM 0.370) because K-Means-Lab is run-to-run non-reproducible (Section 5.6). The full 15-row table is in `code/pics/task3/summary/metrics_table.csv`.)

(Provenance note for the `E:prio_edgef1` row: its numbers were recorded on the original per-region Sobel reference path. The current `impl="sat"` runs Sobel once globally and gives values that differ near region edges (median $\sim 2 \times$ relative difference), so this exact row is not reproducible from the shipped code; the verdict below is robust under both implementations.)

Noise audit — which of these rows can carry a conclusion. Fourteen of the fifteen rows use K-Means-Lab, and that sweep driver never seeds OpenCV’s RNG, so each row is a single unseeded draw. I measured the resulting spread instead of assuming it. Four consecutive runs of one fixed configuration (quadtree, $S_{\text{set}} = [1..32]$, $K = 8, 9,990$ triangles) returned:

metric	run 1	run 2	run 3	run 4	spread
$\Delta E2000$	10.2519	9.8111	9.9255	9.8406	0.441
SSIM	0.36675	0.36030	0.36020	0.36405	0.0066
Edge F1	0.32247	0.30441	0.31731	0.29387	0.029
EPI	+0.04740	+0.04555	+0.04731	+0.04762	0.0021
Quant Error	7.30	7.76	7.38	7.46	0.178

The geometry is deterministic across all four runs (identical 4,995-cell partition); only the palette moves, and eight palette builds on *frozen* geometry returned eight distinct palettes. So each metric has its own noise floor, and a gap only means something when it is large relative to *that metric’s* floor. Comparing a $\Delta E2000$ gap against an Edge F1 gap would not be valid, so the table below uses the matching floor:

comparison	metric	gap	floor	ratio	verdict
K sweep, $K=4 \rightarrow 16$	$\Delta E2000$	2.340	0.441	5.3x	load-bearing
$S_{\text{max}} 32 \rightarrow 64$	SSIM	0.062	0.0066	9.5x	load-bearing
$S_{\text{max}} 32 \rightarrow 64$	Edge F1	0.082	0.029	2.9x	load-bearing
priority, $\Delta \text{MSE} \rightarrow \text{Sobel}$	EPI	0.070	0.0021	34x	load-bearing
priority, $\Delta \text{MSE} \rightarrow \text{Sobel}$	Edge F1	0.107	0.029	3.7x	load-bearing
quadtree vs region-merge	EPI	0.027	0.0021	13x	load-bearing
quadtree vs region-merge	SSIM	0.027	0.0066	4.1x	load-bearing
palette, K-Means-Lab vs Median-Cut	$\Delta E2000$	0.660	0.441	1.5x	marginal — see below
$S_{\text{max}} 32 \rightarrow 64$	$\Delta E2000$	0.260	0.441	0.6x	inside noise
$S_{\text{max}} 64 \rightarrow 128$	$\Delta E2000$	0.069	0.441	0.2x	inside noise
priority, $\Delta \text{MSE} \rightarrow \text{Sobel}$	$\Delta E2000$	0.390	0.441	0.9x	inside noise

So four of the comparisons I draw conclusions from clear their own metric’s floor by $2.9\times$ or better, and three do not. Where a conclusion needed the colour side, I re-ran it with a deterministic palette — that is what Section 5.4 does, and there the $\Delta E2000$ gap is $10.61 \rightarrow 11.77$ (spread 1.16, $2.6\times$ the floor). The quadtree-vs-region-merge colour verdict in Section 5.6 is taken from the *seeded* Table 7, which makes it reproducible, though at +0.32 it is still inside the colour floor; that decision rests on the structure gap instead, as Section 5.6 states. The palette comparison is the interesting borderline case, and I treat it explicitly in Section 5.6 rather than claiming it as a win.

Key findings (all on the single test image `sky.jpg`):

Tying back to Task 2’s motivation. Section 4 reported a negative EPI (≈ -0.03) for the uniform grid and used it as the motivation for adaptive geometry. The adaptive configuration now makes EPI positive for the chosen config (+0.045, Section 5.6) and it rises to +0.109 at $S_{\text{max}} = 64$, while Task 2’s best Edge F1 was 0.233 (the fixed-percentile quantiser). (The two families are not directly comparable: 3 colours vs 16, 9,204 vs 9,990 bricks, but the direction of the change is the one I was after.)

Larger K monotonically improves colour error. From $K = 4$ to $K = 16$, $\Delta E2000$ moves from 11.08 to 8.74 and Quant from 9.25 to 5.62, at a small structure cost (SSIM 0.384 to 0.360).

$K = 16$ minimises colour error, but the best SSIM and EPI of the K sweep both belong to $K = 4$ (0.384 and 0.064), so the chosen config is a balanced compromise.

Raising $S_{\max}$ from 32 to 64 was the largest structure and edge win of the sweep: SSIM moved from 0.359 to 0.421, MS-SSIM from 0.520 to 0.555, Edge F1 from 0.295 to 0.377, and EPI from 0.046 to 0.109 (more than doubled). A coarser start grid leaves more budget for the deep splits where detail matters. The matching $\Delta E2000$ move is $9.95 \rightarrow 10.21$, a +2.6% regression, but that particular gap is smaller than this sweep’s ~ 0.4 noise floor, so I treat the colour side as unresolved here and establish it deterministically in Section 5.4 instead. Going to $S_{\max} = 128$ changes every structural metric by less than 0.01 against 64 (SSIM 0.423 vs 0.421, Edge F1 0.378 vs 0.377) and ends two triangles under budget at 9,988; the colour-side columns move slightly more ($\Delta E2000$ 0.069, Quant Error 0.029, PSNR 0.014 dB) but all of those gaps are an order of magnitude inside the ~ 0.4 noise floor, so the defensible statement is only that 128 buys nothing measurable over 64.

ΔMSE priority is the better choice on the edge-alignment metrics. The Sobel version is the only negative-EPI run of the grid (-0.020) and has by far the worst Edge F1 (0.213 against 0.320), which is the failure that matters here because it means the triangle boundaries stopped following image structure. Both of those gaps are far outside the noise floor, and they are what the choice rests on. Sobel does lead on several colour-side scores ($\Delta E2000$ 9.83 vs 10.22, Quant Error 6.45 vs 7.33, SSIM 0.374 vs 0.370), but those gaps sit inside the noise, so the honest summary is that edge-density allocation over-exploits high-gradient pixels without aligning boundaries to real edges, with no measurable colour-side benefit. The measurements largely rejected this otherwise plausible-sounding suggestion.

Comparing quadtree against region-merge, the quadtree wins on structure (SSIM 0.359 vs 0.332, EPI +0.046 vs +0.019) while region-merge leads on the colour columns ($\Delta E2000$ 9.59 vs 9.95). The structure gap is well outside the noise; the colour gap of 0.36 is not, so I take that verdict from the seeded comparison in Section 5.6 instead. Either way the mechanism is the same and is not in doubt: region-merge collapses to only $\{16,32\}$ cells, while the quadtree keeps a $\{4,8,16,32\}$ mix.

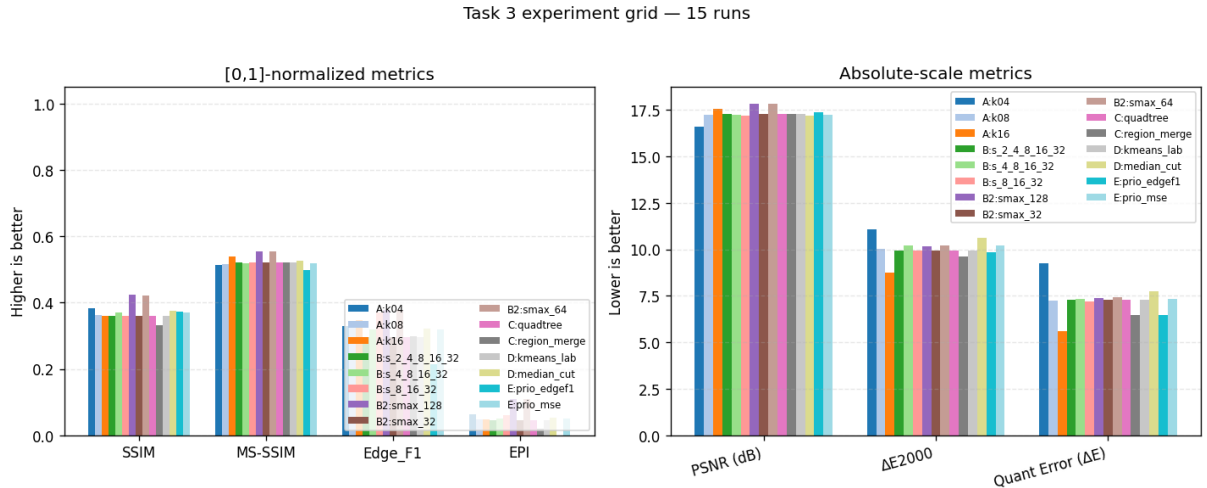


Figure 5. All 15 Task 3 runs as a per-metric bar chart. The x-axis of each panel carries the metrics (left: the four $[0,1]$ -bounded scores; right: the absolute-scale ones), and the 15 runs are series within each group, identified by the legend, which is where the swept variable (K , S_{set} , partition, palette, priority) shows up. The structural-vs-colour split is visible across the whole grid.

5.3 The marginal-benefit / rate-distortion study

Driving $S_{\max}$ upward raised a question I wanted to answer: at what size does splitting stop paying? I ran a dedicated experiment with a 60,000-triangle budget that recorded each split’s marginal benefit (ΔSSE per new triangle) by parent size. Three figures, each from a different file in `code/pics/task3/analysis/`, show the three sides of the answer; the numbers behind them are in Table 4.

parent size	splits	mean $\Delta\text{SSE}/\text{tri}$	share of total benefit
64	484	345,760	39.3%
32	972	111,064	25.4%
16	1766	39,219	16.3%
8	2964	16,889	11.8%
4	3277	8,833	6.8%
2	357	5,071	0.4%

Table 4. Marginal benefit of a split by parent cell size (60,000-triangle run).

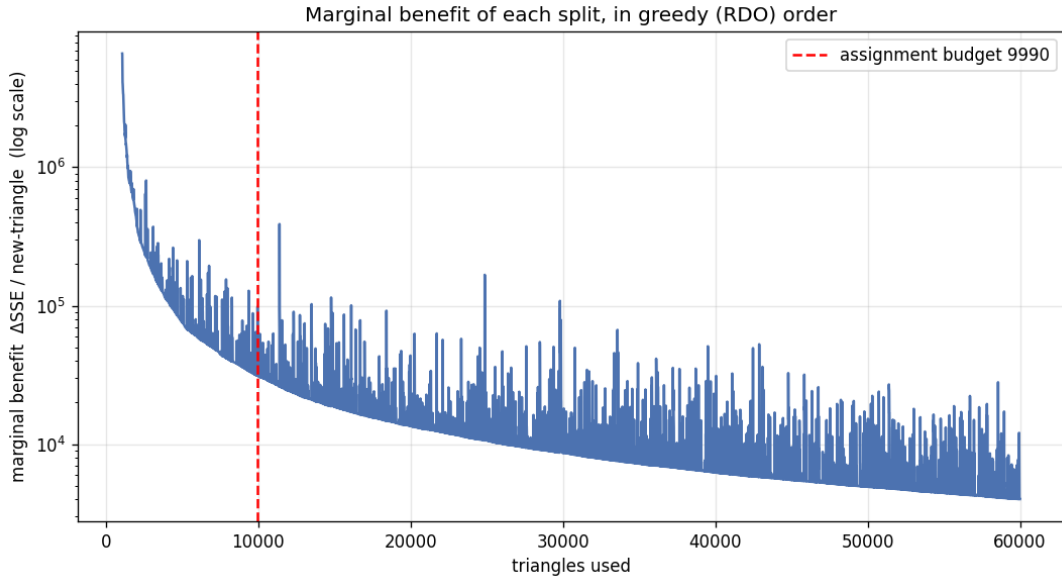


Figure 6. The marginal-benefit curve (`marginal_benefit_curve.png`): each split’s benefit, ΔSSE per new triangle, plotted in greedy split order on a log y-axis against the running triangle count. The envelope decays from roughly 6×10^6 at the first split to about 10^4 at the tail; the red line marks the 9,990 budget, where a split still pays but the remaining budget would only buy ever-smaller cells.

That size-dependence justifies the $S_{\max}$ sweep in the B2 group and explains why a coarse starting grid frees budget for the deep splits that actually matter: the benefit is concentrated in the coarse end of the size range.

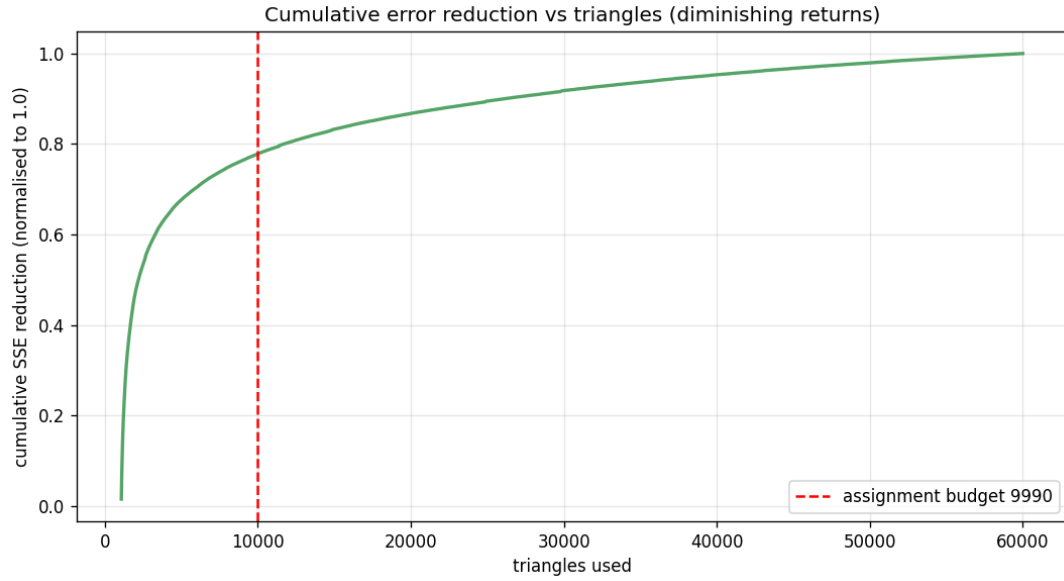


Figure 7. The cumulative view (`cumulative_benefit_curve.png`): the same split benefits added up, normalised to the full 60,000-triangle total. Around three-quarters of the total error reduction is already captured by the 9,990 budget line; beyond it the curve flattens, which is the diminishing-returns half of the story.

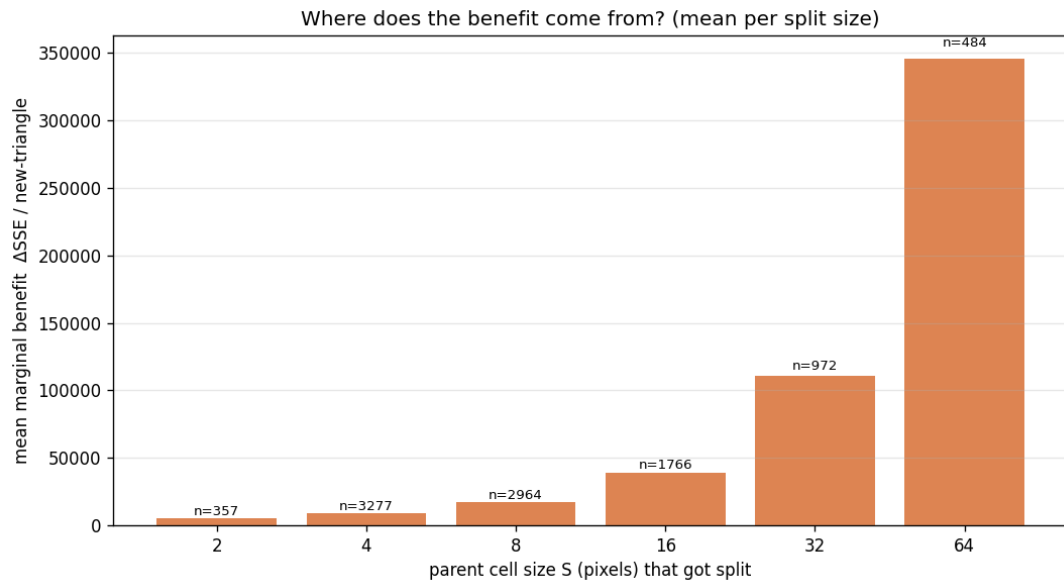


Figure 8. The size breakdown (`benefit_by_size.png`): mean benefit per split grouped by the parent cell size that was split, with the number of such splits on each bar. A size-64 split is worth roughly 68 times a size-2 split, and the large cells dominate because a single coarse split removes a great deal of error, while the many small-cell splits at the bottom of the table together contribute only 0.4% of the total reduction.

5.4 $S_{\max} = 32$ vs 64 — a two-sided trade-off

The metric verdict that $S_{\max} = 64$ wins is only one side of the story. Breaking ΔE down by region and by cell size shows the other side. I re-ran the curve below with a **deterministic Median-Cut palette** ($K = 8$) to isolate the $S_{\max}$ geometry effect from K-Means-Lab’s run-to-run variance, so its values are not directly comparable to the K-Means-Lab rows of Section 5.2 (where $S_{\max} = 32$ gives ΔE 9.95 and $S_{\max} = 64$ gives 10.21). The two sweeps agree in *direction* but differ in magnitude, and the chart is at `code/pics/task3/analysis/smax_curve.png`.

$S_{\max}$	32	64	128	256
ΔE_{2000} (lower better)	10.61	11.77	11.79	11.79
SSIM (higher better)	0.375	0.415	0.415	0.415
Edge F1 (higher better)	0.322	0.403	0.402	0.402

Table 5. $S_{\max}$ sweep (deterministic Median-Cut palette, $K = 8$): perceptual vs structural metrics.

ΔE_{2000} is best at $S_{\max} = 32$ and worsens afterwards, then flattens out. SSIM, Edge F1, and EPI improve up to $S_{\max} = 64$ and then flatten. There is no single best $S_{\max}$: 32 wins on large-area colour and 64 wins on structure and edges, and which one is “optimum” depends on how those two sides are weighted. To test that claim rather than assert it, `code/task3_shadow_crop.py` splits the **source** image into two pixel sets by a luminance percentile (the darkest 40% and the remainder) and averages the per-pixel CIEDE2000 map over each, for both $S_{\max}$ values. Defining the split on the source rather than on either render is what makes the two comparable. Under $S_{\max} = 64$ the shadow set’s mean ΔE rises from 8.37 to 10.90, **+30%**, while the bright set barely moves (12.11 to 12.35, **+2%**) — the degradation is concentrated in the dark regions, which is exactly where a ΔMSE priority has no variance left to split on. Both figures come from that script: `crop_shadow.png` and `crop_sky.png`, with the non-highlighted set dimmed so each figure shows its own pixels. I chose a balanced configuration for this report (Section 5.6).

The mechanism behind the trade-off is that the RDO priority is ΔMSE , which scales with local variance. Edges get split and smooth regions are never split, so a whole large area ends up painted with one colour whose per-pixel error is small but whose *accumulated* perceptual error is large. ΔMSE cannot see large-area uniform drift, but the eye can.

On the two families of metrics above, $S_{\max} = 64$ wins four objective scores while the human eye and the single perceptual-colour metric (ΔE_{2000}) both prefer 32. The only metric that agreed with the eye was the one a metric-only summary would be tempted to drop. The two quantities measure different things: per-pixel structure versus accumulated large-area colour drift. Neither one is “the answer”. That is why Section 5.6 picks a balanced configuration instead of declaring a single winner, and the same episode shows up again as an AI reporting bias in Section 8.1.

$S_{\max}$ sweep on sky.jpg — perceptual vs structural metrics disagree

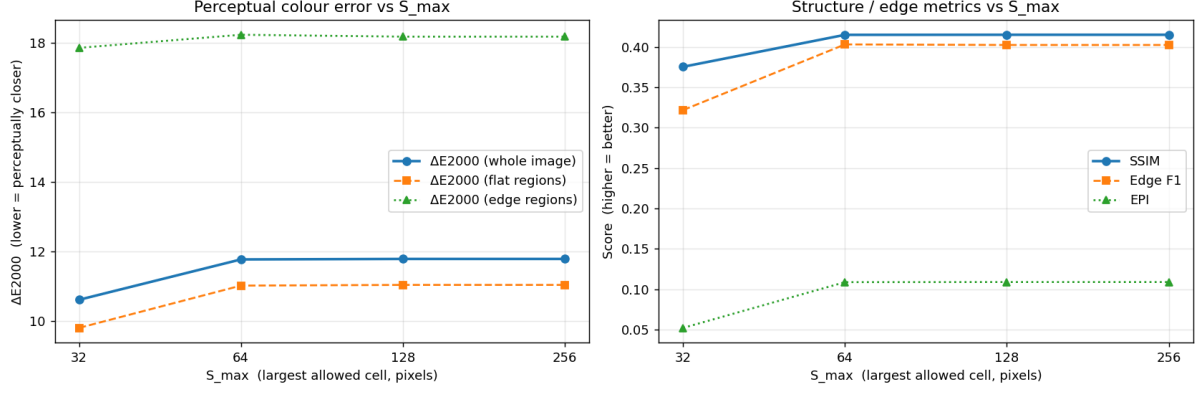


Figure 9. $S_{\max}$ sweep, two panels: perceptual colour error (ΔE_{2000} , best at 32, then worsening and flat) versus structural metrics (SSIM / Edge F1 / EPI), rising to 64 then flat). ΔE_{2000} and SSIM/EdgeF1/EPI point in different directions — there is no single best $S_{\max}$.



Figure 10. The full frame under $S_{\max} = 32$ and $S_{\max} = 64$. The panels are pixel-aligned and the non-shadowed pixels are dimmed, so the left pair shows the source against the $S_{\max} = 32$ render and the right pair against the $S_{\max} = 64$ render. Mean CIEDE2000 over the shadow set goes $8.37 \rightarrow 10.90$ (+30%) while the bright set moves $12.11 \rightarrow 12.35$ (+2%); the bright-set companion is `code/pics/task3/analysis/crop_sky.png`. Both figures are written by `code/task3_shadow_crop.py`.

5.5 Problems found and solved

The first problem was a quadtree inverted-loop bug that the AI introduced. The original code guarded `if n_tri <= budget: return` and looped `while n_tri > budget`, which reflected the wrong mental model: splitting *increases* the count, so the loop must run *while* budget remains. On `sky.jpg` the quadtree never split, using only 4,320 of 9,990 triangles. I fixed it to `while heap and n_tri + 6 <= budget`, and 945 splits then reach exactly 9,990 triangles and EPI turns positive (-0.043 to $+0.049$). I recorded this as an AI-introduced bug in Section 8.2.

The second problem was region-merge candidate alignment. Merge candidates generated at every half-pixel offset landed off the target-size grid and could never form the next level’s 2×2 blocks, so the run stalled at 8,640 merges. I fixed this by generating candidates aligned to the `target_size` grid, after which `region_merge` reached 9,990 triangles in about 1.7 s with 44,415 merges.

5.6 Chosen configuration

I went with a balanced choice, near-best perceptual colour without throwing away structure. The driver is `code/task3_best.py`, and the visuals are at `code/pics/task3/best/best_config.png` and `best_compare.png`.

setting	value	why
partition	quadtree	keeps a $\{4,8,16,32\}$ mix; near-best ΔE with much better structure than region_merge
$S_{\max}$	32	best ΔE_{2000} ; 64 improves structure but worsens large-area colour
$S_{\min}$	1	no-op on this image (the budget binds at size 4; the B group in Section 5.2); kept to allow fine cells where the budget permits
K	16	best ΔE_{2000} + quant error
priority	mse ($\Delta \text{MSE} / 6$)	wins on the structure/edge metrics (see Section 5.2)
palette	kmeans_lab	wins the colour columns (ΔE_{2000} , Quant Error, PSNR) but loses all four structural ones to median_cut , and its ΔE_{2000} edge is only $1.5\times$ the noise floor — a genuine trade-off, resolved in favour of colour because that is the chosen config’s headline claim; see Section 5.6

Table 6. The chosen Task 3 configuration.

Metrics (9,990 triangles, `sky.jpg`):

config			ΔE_{2000}	SSIM	PSNR	Edge F1	EPI
quadtree	$S_{\max}=\mathbf{32}$	$K=\mathbf{16}$	8.95	0.359	17.56	0.346	+0.045
kmeans_lab (chosen)							
region_merge	$S_{\max}=32$	$K=16$	8.63	0.339	17.59	0.298	+0.015
kmeans_lab							
region_merge	$S_{\max}=32$	$K=16$	9.47	0.338	17.54	0.336	+0.016
median_cut							

Table 7. Metrics of the chosen configuration and its two rivals (seeded, from `best/*_metrics.json`).

The palette choice is the one setting where I am **not** claiming a clean win, so it is worth stating plainly. The D-group holds geometry fixed and varies only the palette, and on that comparison Median-Cut is *better* on every structural metric — SSIM 0.375 vs 0.359, MS-SSIM 0.526 vs 0.520, Edge F1 0.322 vs 0.295, EPI +0.052 vs +0.046 — while K-Means-Lab is better on the colour

columns (ΔE_{2000} 10.61 vs 9.95, Quant Error 7.76 vs 7.30, PSNR 17.17 vs 17.28). Only the ΔE_{2000} gap clears the noise at all, and it clears it by $1.5\times$, so on the perceptual-colour metric the two are close to indistinguishable. I chose K-Means-Lab because the chosen configuration’s headline claim in this report is perceptual colour fidelity, and because a K-Means palette warm-started from a fixed seed is reproducible while remaining free to place centroids where the data actually sits — Median-Cut can only split axis-aligned boxes. But a reader who weights edge alignment or EPI above colour error should pick Median-Cut, and the evidence for that preference is at least as strong as the evidence against it. I would rather flag that than present a $1.5\times$ margin as a result.

(Numbers from `code/pics/task3/best/*_metrics.json`, regenerated by `task3_best.py` with `cv2.setRNGSeed(0)`; these JSON files are the single source of truth for the chosen config. The same configuration appears in three places in this report with slightly different ΔE_{2000} values — 8.74 (Section 5.2 A:k16, unseeded sweep), 8.95 (this table, seeded), 9.03 (`docs/progress/Task3.md` §9, earlier run). All three are “the same configuration” measured under K-Means-Lab’s run-to-run non-reproducibility, which I quantified rather than assumed: re-running one fixed configuration three times gave $\Delta E_{2000} = 10.25 / 9.93 / 9.81$, a spread of 0.44, and building the $K = 8$ palette eight times on frozen geometry returned eight distinct palettes. The noise floor is therefore $\sim 0.4 \Delta E_{2000}$. The conclusion is unaffected, because it rests on the structure gap (Edge F1 0.346 vs 0.298, EPI +0.045 vs +0.015), which is 3–10 $\times$ the colour spread — see Section 5.2’s noise audit.)

I also ran a border-free ablation, reproducible with `python code/task3_best.py -no-border`, which re-runs the chosen configuration with only the 1-px boundary pass suppressed — same partition, same palette, same labels. All numbers above (and in Sections 4–5) include that border, which on the chosen Task 3 render covers 11.6% of pixels — the third most common colour there, behind two flat-sky palette entries at 19.2% and 14.8%, and therefore a large, constant error term. With the border suppressed the metrics improve markedly: PSNR 23.35 dB, SSIM 0.571, ΔE_{2000} 6.83, Edge F1 0.507, EPI +0.274. The border is a large and unevenly distributed penalty: PSNR rises by 5.79 dB and EPI by $6.0\times$ when it is removed, while quantisation error does not move at all (5.7898 in both runs), because that metric never looks at the rendered image. Because every compared run carries the same border, the *comparisons* and rankings in this report are unaffected. The borderless render is the colour-only reading that the drawn boundaries should not be weighed as brick colours, and I keep the bordered numbers as primary because they are the actual delivered output.

Even though `region_merge` achieves the lowest ΔE of the three configurations in Table 7 (8.63), I did not pick it because it merges all fine cells away, leaving final sizes of only {16,32}. That gives it the worst structure score of those three (Edge F1 0.298, EPI +0.015). Across the whole 15-run sweep one row is worse on Edge F1 — the Sobel-priority run at 0.213, discussed in Section 5.2 — but that run fails on edge alignment rather than on cell-size coverage, which is the property being compared here. The quadtree keeps a {4,8,16,32} mix and costs +0.32 ΔE against `region_merge` for a much better structure score. The comparison is reproducible: `task3_best.py` seeds OpenCV’s RNG and writes per-config metrics JSON to `code/pics/task3/best/*_metrics.json`.

One caveat on that +0.32: seeding makes it reproducible, but 0.32 is still below the $\sim 0.4 \Delta E_{2000}$ noise floor measured in Section 5.2, so on colour alone the two are not separated by this experiment. The decision rests on the structure gap (Edge F1 +0.048, EPI +0.030), which is reproducible because it is a property of the cell-size distribution rather than of a particular K-Means local optimum.

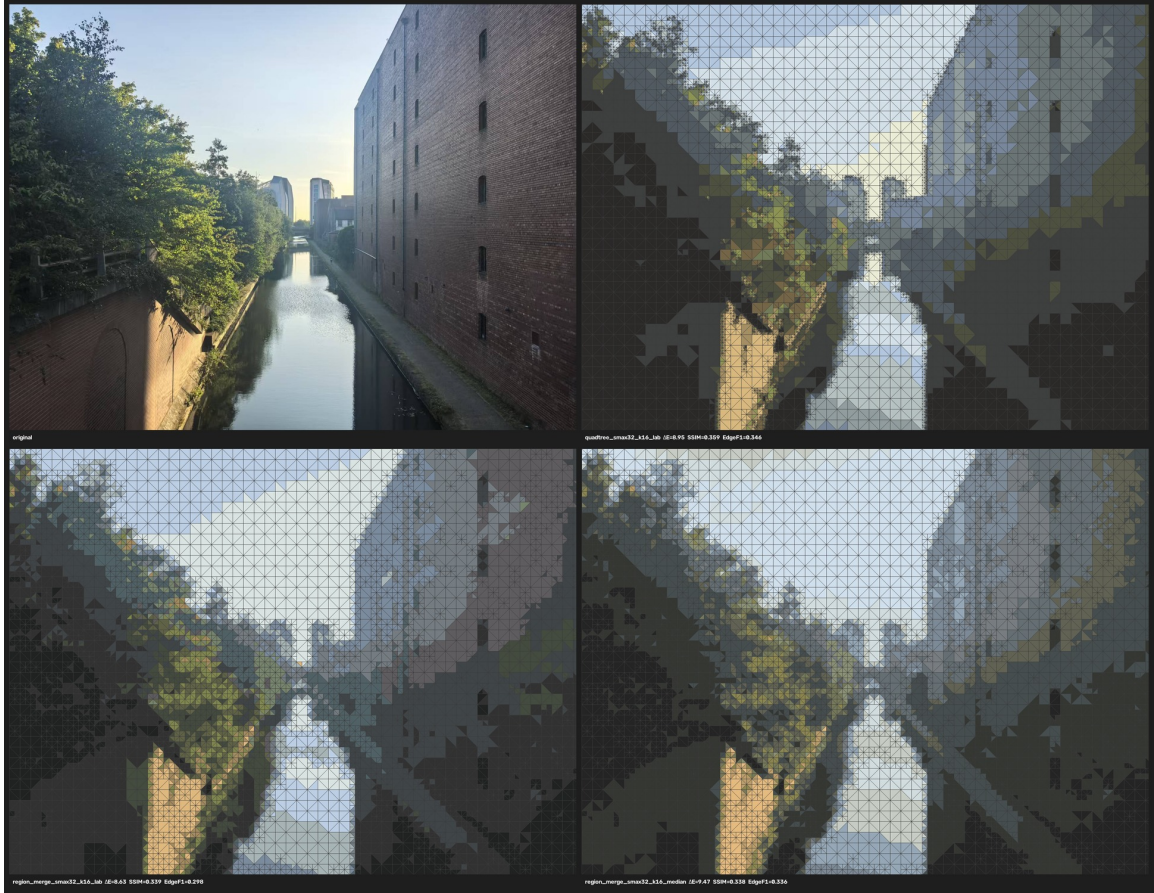


Figure 11. The chosen configuration against its two region-merge rivals (original + three renders, key metrics per panel). The quadtree render keeps fine cells in detail regions that region-merge flattens.

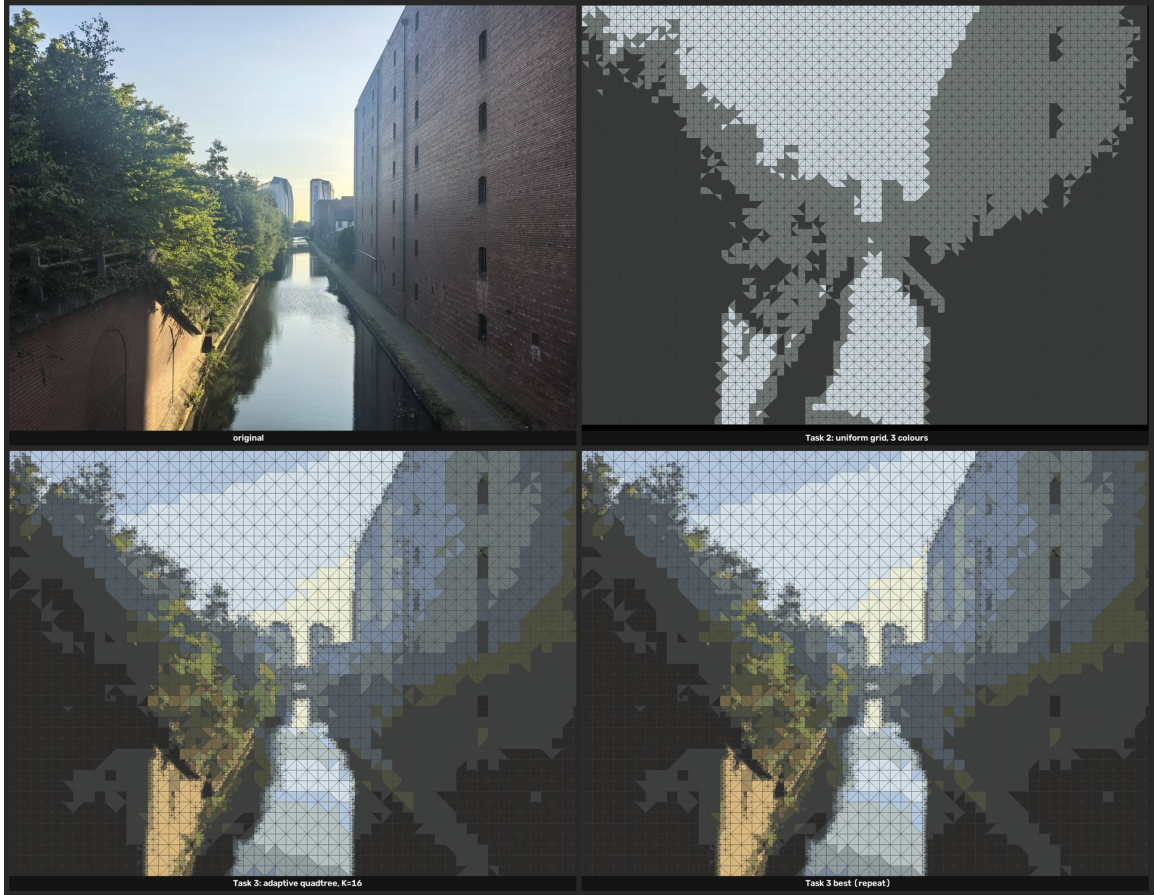


Figure 12. Task 2 (uniform grid, 3 colours) versus Task 3 (adaptive quadtree, $K = 16$) on the same image: the adaptive tessellation concentrates small bricks on detail and large bricks on flat regions.

5.7 Brick-size summary

Besides the rendered image, the system also outputs a brick summary with the total number of bricks and the count for each brick size. The chosen configuration from Section 5.6 produces the following partition, written by `code/task3_best.py` to `code/pics/task3/summary/brick_size_counts.json`.

cell size (px)	cells	bricks (=2× cells)
4	724	1,448
8	1479	2,958
16	981	1,962
32	1811	3,622
total	4,995	9,990

Table 8. Brick-size summary of the chosen configuration (Task 3 deliverable).

The quadtree reaches four brick sizes, so multi-size coverage is achieved with a true $\{4,8,16,32\}$ mix. The per-size histogram of the $K = 16$ sweep run is at `code/pics/task3/A_ksweep/k16_size_hist.png`, and the JSON above is the count source.

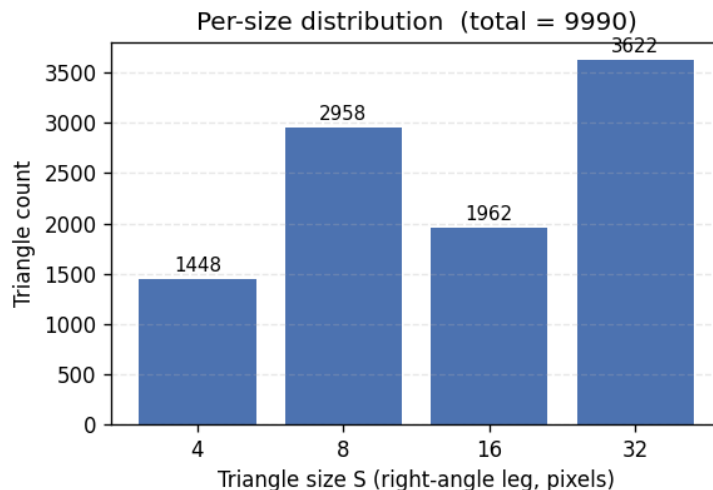


Figure 13. Brick-size histogram of the chosen ($K = 16$) quadtree run: a $\{4,8,16,32\}$ mix with the most cells at size 8 and 32.

6 Task 4 — real-world camera scenario

6.1 What this task had to solve

When I first ran Task 3’s offline pipeline on the 1706×1279 reference image, partition plus means plus render plus metrics came out at roughly 0.1 FPS, so the speed question was already in the room before Task 4 began. Task 4 takes the chosen config from Section 5.6 and drops it into a camera loop at 640×480 , which is where the speed and stability problems the offline setting hid become the central engineering problem I had to solve.

Running the same pipeline live changes the problem in two concrete ways. First, a frame has to

be finished before the next one arrives, so the per-frame cost now has a hard ceiling that the offline runs never faced. Second, the output has to stay stable from one frame to the next: the mosaic must not visibly re-organise itself every frame just because the sensor noise moved by a few grey levels. Both constraints were invisible in the offline setting, where each run was a single image with no deadline and no memory of the previous frame, and both turned out to be the actual engineering content of this task.

I took real-time processing as the main thread, because making the pipeline fast is also the natural way to find out what is actually wrong with it. Profiling first and fixing only what the profile indicts turned out to be the method that carried the whole task: every change in Section 6.4 came from a measured bottleneck, and every one of them is paired with the before/after numbers that motivated it. The deliverable is `code/camera_app.py`, a live loop that opens the camera at 640×480 with `CAP_DSHOW`, runs the full pipeline per frame, and shows input and render side by side in a resizable window with a compact HUD (q quits, s snaps, p pauses, r resets). I also built a headless `-no-show -input <clip>` mode so that every number in this section can be reproduced without a camera; `code/task4_verify.py` is the assertion suite and `code/task4_verify_util.py` holds the helpers.

6.2 Measured baseline (before any optimisation)

The full pipeline, before any optimisation, is the diagram below. Its seven timed stages carry the millisecond figures; the other two boxes are the resize and the display, which are not timed. Each timed stage is where one of the optimisations in Section 6.4 lands, so the diagram doubles as the map of the changes that follow: the two red stages were the measured bottlenecks.

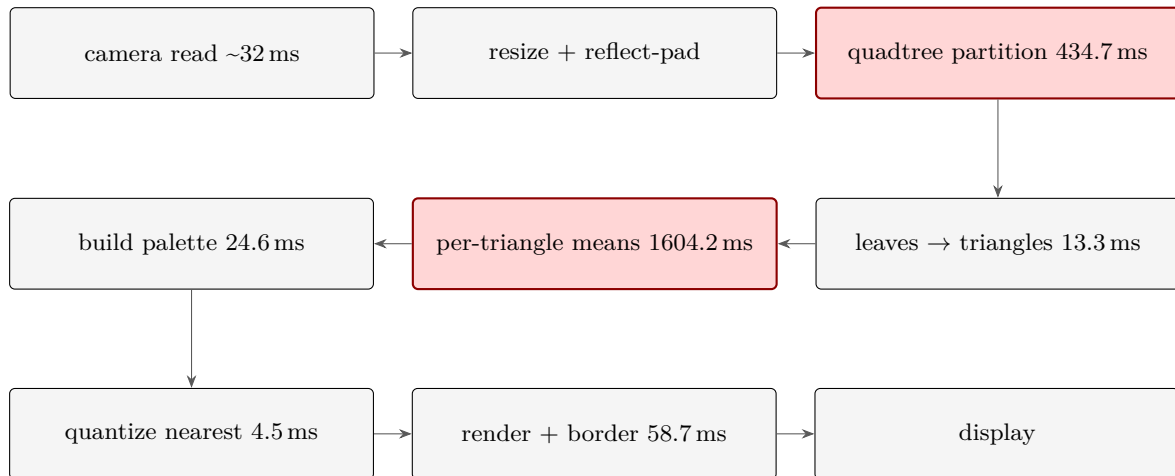


Figure 14. The full pipeline before any optimisation. The two shaded stages are the measured bottlenecks and account for 95% of the frame; the per-stage figures are in Table 12.

The baseline is measured at the full-quality config (quadtree, $S_{\text{set}} = [1..32]$, $K = 16$, `kmeans_lab`, `budget=9990`, 640×480 , `scale=1.0`):

`triangle_means_bgr` dominates because per triangle it zeroes a full-frame mask and calls `cv2.mean` — cost is $O(T \cdot H \cdot W)$. The per-triangle cost scales with image area (903 μs /triangle at 1706×1279 , 143 μs at 640×480 , 39 μs at 320×240 — area ratios $7.1 \times$ and $4.0 \times$ track the per-triangle ratios $6.3 \times$ and $3.6 \times$, which is the $O(T \cdot H \cdot W)$ prediction). Two cheap parameter levers alone reach ~ 9 FPS (reduce scale to 0.4 and budget to 1000, $K = 6$), so the optimisation work had to beat **9 FPS at 1000 triangles**, not the easier 0.45 FPS. A later correction established that `build_palette` was not $12 \times$ slower. A 2-frame run made a one-time ~ 0.5 s warm-up look like a per-frame cost (median of two samples is their mean); with 30-frame runs the stage is

Stage	median ms	mean ms	Share
camera read	~32	—	—
quadtree_partition	434.7	427.4	20%
leaves_to_triangles	13.3	13.5	1%
triangle_means_bgr	1604.2	1595.3	75%
build_palette	24.6	105.0	1%
quantize_nearest_bgr	4.5	4.9	0.2%
render_triangles	58.7	58.4	3%
frame total	2140	2204	0.45 FPS

Table 9. Task 4 measured baseline before optimisation (full-quality config, 640×480 , 9,990 bricks). The six processing stages sum to 2,140ms, which is the frame total the FPS column is computed from; the ~32ms camera read is measured separately and is not included in that sum. The stage shares are percentages of the 2,140ms processing total.

3–25ms/frame. I learned not to read per-stage percentages off a run too short for the statistic to mean anything. Frame counts per table: the 0.45-FPS baseline is an 8-frame run; the corrected palette figures are 30-frame runs; per-stage isolation figures are single-frame-repeat or 12-frame paired runs (details in `docs/progress/Task4.md`).

6.3 Camera capture and display

`camera_app.py` opens the real camera, runs the full pipeline per frame, and shows a side-by-side window. Before trusting the display I checked the saved frames numerically, on `code/pics/task4/L1_baseline/frame0001_{input,render,compare}.png` — that is a different frame from the one in Fig. 15, and I keep the two apart deliberately, because the numbers below are measurements of that frame and would be wrong to attach to another. On it: the render has exactly 17 distinct colours ($K = 16$ palette + 1 border colour); the border (60,60,60) covers 74,592px, or 24.3% of the 640×480 frame, which makes it the single most common colour in the render and measurably affects metrics, so the render path is the only place a border is drawn; the palette spans the full dark-to-bright range with a real spread of hues; and overall contrast is broadly preserved (render std 62.1 vs input 70.4, measured on the saved frames). One defect caught me during this stage: the HUD’s FPS first read its timestamp immediately after `cap.read()`, so it reported the camera’s own ~33FPS no matter how slow the pipeline was. The timestamp is now taken after all per-frame work, and the HUD, the headless log and the benchmark share one measurement path.

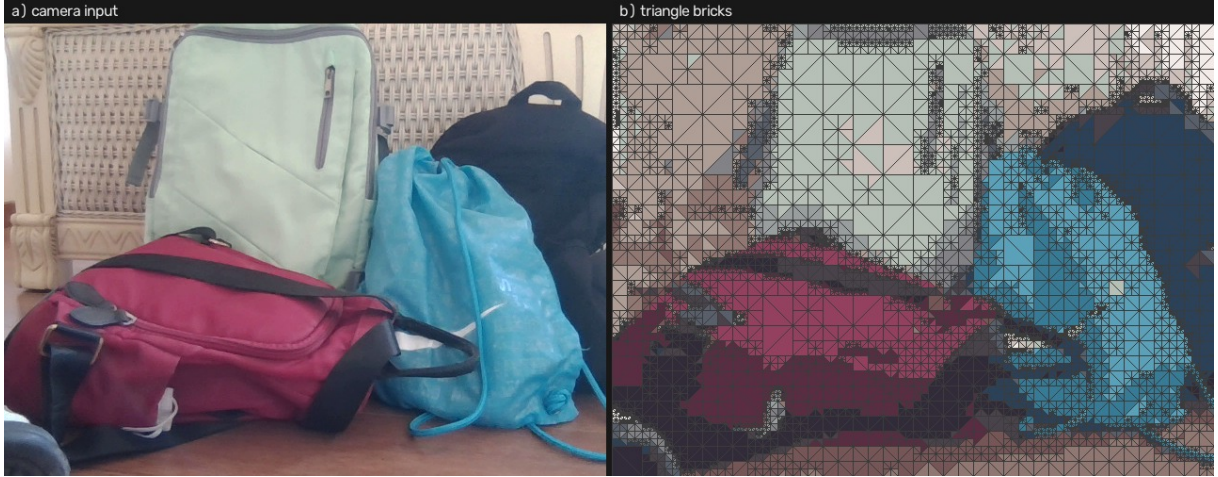


Figure 15. A real camera frame beside its triangle-brick render ($K = 16$, 9,990-brick quality preset), shown at the 640×480 the loop runs at. Input and render appear together because a mosaic judged without its input says nothing about fidelity. This scene is the clearest single demonstration of the two adaptive decisions working together: the partitioner puts its smallest cells on the bag seams, zips and handles while the flat wall behind stays in large coarse cells, and the 16-colour Lab palette keeps the sage-green, teal and maroon cases apart instead of collapsing them into one lukewarm average. The two panes are not the same brightness, though: the render is more vivid than the input, because a brick mean is assigned the nearest palette entry and the nearest entry to a saturated colour is a more saturated one. The consistent colour artefact I discuss below is on the portraits, not here.

6.4 Meeting the real-time constraint

Step 1: mean colour. The shipped `triangle_means_bgr` has a real defect, not just a speed problem: `cv2.fillPoly` paints a one-pixel fringe along the *right* and *bottom* edges of every triangle (pixels whose centres belong to the neighbouring cell), so every Task 2/3 brick’s colour is contaminated by its bottom-right neighbours. The leak is up to 400% on size-1 bricks and around 8% of pixels overall. I added two new implementations in `code/brick_means.py`:

-means	FPS	means stage	whole frame	ΔE_{2000}	PSNR
mask (shipped)	0.41	1716 ms	2260 ms	9.947	14.90 dB
fast (exact in-cell)	1.35	20.8 ms	532 ms	8.633	15.33 dB
sample (9 interior pts)	1.47	5.2 ms	493 ms	10.244	14.87 dB

Table 10. Mean-colour implementations: speed and quality (640×480 , 9,990 bricks).

fast is $82\times$ faster on the stage, $3.3\times$ overall, *and* closer to the source ($\Delta E\ 9.95 \rightarrow 8.63$) because it averages only in-cell pixels. **sample** is fastest but worse than even the leaky reference on small bricks (nothing to sample inside a 1–4px triangle). The defect is not a corner case: over the real 4,995-cell partition the reference averages 8.1% foreign pixels (186,021 of 2,297,721), and per-triangle means differ from the correct in-cell means by an average of ~ 3.0 grey levels per channel (max 46.1). Every recorded Task 2/3 number inherits this small directional bias, but cross-run comparisons are unaffected because all runs used the same leaky reference; the corrected in-cell means are what the report’s Task 2/3 numbers would give if the fringe were absent — a disclosed limitation. (Quantified with `compare_means` in `code/brick_means.py`; full numbers in `docs/progress/Report_prep_assets.md` §3.)

Step 2: partition. `quadtree_partition` recomputed a numpy region mean per split candidate. Two fixes: drop 4 dead SSE computations per split ($436 \rightarrow 326$ ms), then replace numpy

region means with **summed-area tables** ($O(1)$ rectangle SSE), batched so the $\sim 160,000$ small per-candidate calls collapse into one query per split. Isolated:

Implementation	2000 bricks	5000 bricks	9990 bricks
ref (numpy)	80 ms	173 ms	326 ms
sat (tables, batched)	44 ms	69 ms	107 ms
speedup	1.8 $\times$	2.5 $\times$	3.0$\times$

Table 11. Partition implementation: summed-area tables vs numpy reference.

The partition is **bit-identical** under **mse** priority (cell set and size distribution match exactly), so no Task 3 result is invalidated. Caveat: **edgef1** priority with the table impl uses a global Sobel map and is NOT numerically equal to the recorded per-region-Sobel runs (median $\sim 2\times$ relative difference near region edges); the recorded E_priority numbers are pinned to `docs/progress/Task3.md` and `metrics_table.csv`, and the code prints an explicit note when this combination is used. The conclusion (Δ MSE beats Sobel priority) is robust under both implementations.

Step 3: whole-pipeline analysis. With means and partition fixed, the frame is ~ 216 ms and the remaining bottlenecks are partition ~ 107 ms ($\sim 50\%$, 1,565 greedy splits that add 4,695 cells to the 300-cell start grid, with the cost dominated by numpy-call count rather than the arithmetic), render ~ 55 ms ($\sim 25\%$, $\sim 20,000$ Python-to-cv2 crossings), palette ~ 23 ms ($\sim 10\%$, per-frame K-Means), and triangles ~ 13 ms. I applied four optimisations (all exact unless noted). Optimisation A is a batched render that groups up to K `fillPoly` calls by colour plus one `polylines`, taking render from 55 ms to ~ 8 ms with **0 differing pixels**. Optimisation B precomputes split priorities by scoring all $\sim 102k$ candidates up front so the greedy loop is pure Python; partition drops from 107 ms to ~ 30 ms and the leaves stay bit-identical. Optimisation C changes the palette refresh cadence to rebuild every 10 frames, keeping the palette byte-stable in between, which amortises palette from 22.0 ms to 4.9 ms per frame. Optimisation D vectorises `leaves_to_triangles`, taking triangles from 13 ms to ~ 2 ms. Paired in-process benchmark (synthetic 640×480 , reps=5):

stage	old	new	speedup
partition	72.3 ms	33.8 ms	2.14 $\times$
triangles	8.3 ms	2.2 ms	3.79 $\times$
render	28.1 ms	8.4 ms	3.35 $\times$
palette (amortised)	22.0 ms	4.9 ms	4.47 $\times$
whole frame	129.0 ms	59.5 ms	2.17$\times$ (~ 16.8 FPS processing-only)

Table 12. Whole-pipeline paired benchmark after optimisations A–D (synthetic 640×480 , reps=5).

App-level before/after (same machine, same input): TOTAL $130.5 \rightarrow 62.0$ ms with ΔE_{2000} $11.014 == 11.014$, PSNR $15.68 == 15.68$, **0 pixels differ**.

Step 4: frame-to-frame jitter (root cause). With the camera held still the mosaic still flickered every frame. To find out why, I probed headlessly in `code/task4_verify.py`: the pipeline was driven on four synthetic sequences (identical frames, noise $\sigma = 2$, frozen geometry, and a +1.5-brightness-per-frame ramp), and on each sequence the per-frame canvas churn was measured both on ordinary frames and on the frames where the palette rebuilds (every 10th). Churn here means the fraction of pixels whose maximum channel change exceeds 8 grey levels. The probe results are the source for every number in this step:

sequence	per-frame churn	on a palette-rebuild frame
identical frames	0.000%	canvas 76.6%
noise $\sigma = 2$	0.55–0.96%	canvas 76.7%
noise, geometry frozen	0.31–0.59%	canvas 76.7%
brightness drift	2.85–8.07%	canvas 76.7%

So ordinary frames churn by at most a few percent, but on the rebuild frame the canvas jumps by 76.7% of its pixels. Three independent causes came out of the probe.

The dominant one is K-Means instability. A rebuild re-runs `cv2.kmeans` from OpenCV’s global RNG, which lands on a different local optimum each time, and because every triangle’s label is re-decided against the new palette, a different optimum re-colours most of the canvas at once. Feeding identical pixels through K-Means without re-seeding gave a max palette delta of **180** (the probe’s `pal_d1` column); with a fixed seed, identical input collapses the delta to 0, but two different noisy frames with the same seed still produced a delta of **144**. Re-seeding alone does not solve this, so I had to attack the rebuild itself rather than its random seed.

The second cause is that the pipeline had no temporal coherence anywhere. Partition, means, and quantise were recomputed every frame from noisy pixels, so sensor noise was moving near-tie split decisions on 3–5% of cells per frame and flipping labels near palette boundaries on roughly 3% of cells. With nothing to anchor the output, even a steady scene drifted.

The third cause is camera auto-exposure and white-balance drift, which amplifies the other two. A 1.5/255-per-frame brightness ramp was enough to raise per-frame canvas churn from about 0.5% to 2.9–8.1%. So even after I fixed K-Means and added coherence, an unlocked camera exposure would still keep churning the output.

Step 5: the fix (A+B+C). Fix A is to warm-start the palette from the previous one, refining the previous local optimum instead of jumping to a new one, plus a dead-band that discards a rebuild where no entry moves by more than 2 grey levels. Fix B is to reuse the partition when a keyframe signature says the scene has not changed; static frames keep the same leaves, while means are still re-extracted from the live pixels. Fix C is quantise hysteresis: a per-triangle label memory keeps the previous label unless a different palette entry wins by a relative margin. Measured (churn = fraction of pixels whose max channel change exceeds 8 grey levels):

metric	before	after
palette delta on rebuild, identical input	153	1
steady per-frame churn, noise $\sigma = 2$	2.400%	0.064% (37.7× less)
rebuild-frame churn, identical frames	0.013%	0.000%
rebuild-frame churn, noise $\sigma = 2$	0.029%	0.000%
app static TOTAL ms (temporal OFF → ON)	68.9	27.5
partition stage ms	37.7	0.2 (reused)

Table 13. Frame-to-frame jitter fixes (temporal coherence A+B+C): before/after.

The rebuild-frame flash measured in Step 4 (76.7% of pixels on every 10th frame) is gone — a rebuild is now a refinement, not a re-colouring; the continuous crawl drops $\sim 38\times$ on realistic sensor noise; and the partition stage disappears on a static scene. Note the two palette-delta numbers: Step 4’s **180** is the palette-only micro-test (`pal_d1` in the probe, no re-seed), while Table 13’s **153** is the app-level cold-vs-warm rebuild measured at the temporal layer — they are

two different probes, and Table 13 compares the fix to the second of them.

Step 6: render reuse and row-run means (opt E). Two remaining per-frame costs on a static scene: means (~ 14 ms) and render (~ 10 ms). E1: exact **row-run means** — each triangle half is a contiguous pixel run per row, so a per-row prefix-sum table answers a row’s run in two lookups; bit-identity is provable (exact integer sums $< 2^{24}$, so float32 holds them exactly and addition order cannot change the value). `task4_verify` asserts $\max |\text{diff}| = 0$: $14.2 \rightarrow 7.7$ ms ($1.85\times$). E2: **render reuse** — the canvas is a pure function of (`tri`, `labels`, `palette`, `shapes`); when geometry is reused and labels+palette are byte-identical, the cached canvas is the image, so rasterising is skipped ($10 \rightarrow 0$ ms on a static scene; under noise it correctly does not fire, because labels change). Stacked effect on a static scene: 68.9 ms baseline $\rightarrow$ **27.5 ms** with temporal coherence (step 5) $\rightarrow$ **11.4 ms** with opt E as well, i.e. $\sim 6\times$ overall, all exact reuse and no quality cost. Note that the 37.1 FPS quoted in step 8 below is the *step-5* figure ($1/27.5$ ms), measured before opt E existed; the delivered static number is 11.4 ms, about 88 FPS. I have left both in place because each is a real measurement of the build that existed at the time, but they are not comparable and I should not have quoted 37 FPS as the end state.

Step 7: display panel. Two presentation defects fixed: the HUD previously drew 14 lines on solid black boxes covering $\sim 35\text{--}40\% \times 30\text{--}35\%$ of the render pane (now three lines on one translucent panel, $\sim 15\% \times 3\text{--}4\%$); and resizing a `WINDOW_NORMAL` window stretched the bricks (fixed by uniform letterbox scaling into the window’s image area, so OpenCV’s stretch becomes the identity).

Step 8: reuse freezes colour + camera AE/WB lock. Even after A+B+C, a static scene still showed residual colour churn (0.101% of pixels/frame) because means and quantize were still re-running every frame. The fix is, when the partition is reused, also freeze the previous labels plus the canvas, which makes an unchanged scene byte-identical frame to frame. I also locked the camera’s auto-exposure (`-lock-ae`, optionally at a fixed `-exposure`) and its auto white-balance (`-lock-wb`, a separate flag) because a steady exposure is what makes freezing colour safe rather than stale. Both are opt-in flags, not preset behaviour — the `quality` preset sets only `scale`, `budget` and `K`, so a reader running the default gets drifting exposure unless they pass the flags, and the same is true of the failure case below. Measured: churn 0.101% $\rightarrow$ **0.000%**, static effective FPS 22.6 $\rightarrow$ **37.1**. I confirmed on the real camera that the jitter is noticeably reduced and the fidelity is no worse.

6.5 Behaviour across different content

The same pipeline was run, with no per-scene tuning, on seven camera snapshots captured with `camera_app.py -lock-ae` into `code/pics/task4/` (`snap_000.png` ... `snap_006.png`, each pairing the raw frame on the left with its render on the right). I did not pick these as a showcase set; I picked them so that each one breaks the pipeline in a different way, which is the only way to learn what a fixed configuration actually survives. Table 14 states what each scene is for, and the figures below are read against that column rather than as a gallery.

scene	content	the pipeline property it puts under stress
snap_000	portrait against a flat wall	large flat region next to fine detail: does the partitioner actually spend its budget where the content is?
snap_001	close-up portrait, curtain behind	dense curved structure at close range: do brick boundaries follow a face contour, or drift off it?
snap_002	mechanical keyboard on a desk	high-frequency regular texture: is the smallest cell size fine enough, and does the palette keep neighbouring keys apart?
snap_003	saturated luggage against a warm sofa	hue rather than luminance variation: can a $K = 16$ palette separate distinct hues, or average them?
snap_004	desk lamp in a dim room	extreme dynamic range: does a saturated light source survive, and what happens to the shadows around it?
snap_005	sheer curtain against a window	structure carried by shading rather than hue: can brick density alone carry a fold pattern?
snap_006	dim wall beside a bright window	large low-contrast gradient: this is the weak case, and I treat it as such below

Table 14. The seven-scene sweep and the failure mode each scene was chosen to probe. Every scene runs the identical configuration chosen in Section 5.6.

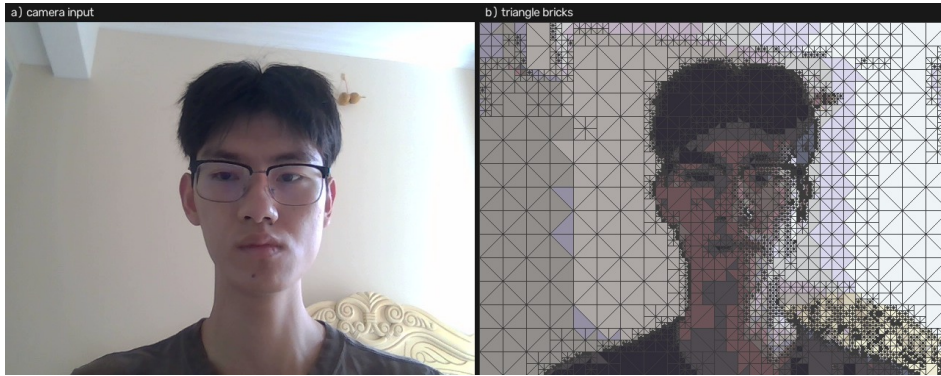


Figure 16. Portrait against a flat wall. The partitioner spends its small cells on the head: the hair mass, the glasses frame and the jaw edge are resolved with a dense fine tessellation, while the cream wall behind carries the largest cells in the frame. The brick count is the same 9,990 as every other scene — the geometry moved to the content, not the budget.

Two things are visible in **snap_000** and **snap_001** that I want to name rather than gloss over. First, it works: the glasses outline, the face contour and the hair edge all survive as dense fine bricks, and in **snap_001** the close-up shows the brick boundaries tracking the cheek and temple curve rather than cutting across them, which is the edge-alignment property EPI measures in Section 5.2. Second, there is one consistent colour artefact across both portraits: the cool-toned indoor light comes out warmer than it went in, because the palette assigns a skin tone to a cool-lit face from a palette built on other frames. A white-balance pass before quantisation would fix it, and I did not add one, so it is a real limitation of this build rather than a rounding detail.

The remaining scene, the luggage pair in Fig. 15, is the case that separates palette from geometry: saturated hues against a warm background give the $K = 16$ palette genuinely distinct colours to separate, and the partition shrinks onto the seams. The pattern across all seven is consistent: the pipeline is limited by the palette, not by the geometry, wherever the content is close to a colour the palette does not contain — which is a different claim from the Task 3 result, where the limit was a spatial one.

The palette was reused across frames within tolerance (`-palette-refresh 10`); no per-scene recalibration was needed, and no scene in the sweep used a different configuration.

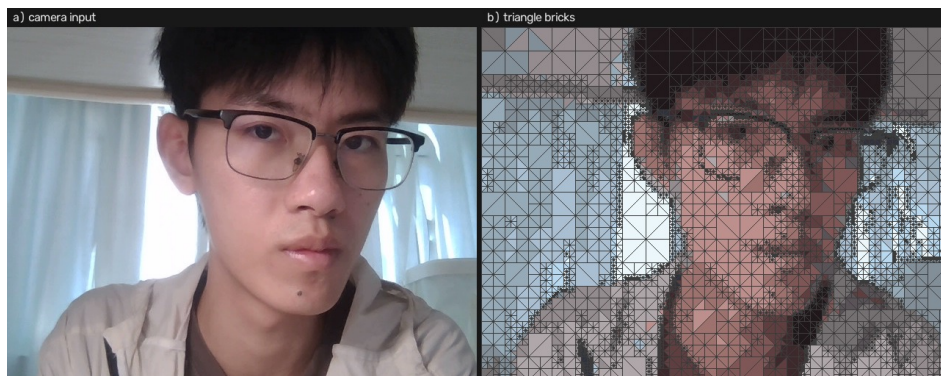


Figure 17. Close-up portrait with a curtain behind. This is the hardest case for boundary placement: the subject is large, well lit and fills the frame, so any systematic drift between the brick boundaries and the face contour would be immediately visible. The fine cells cluster on the glasses, eye sockets and jaw, and the curtain behind stays coarse.

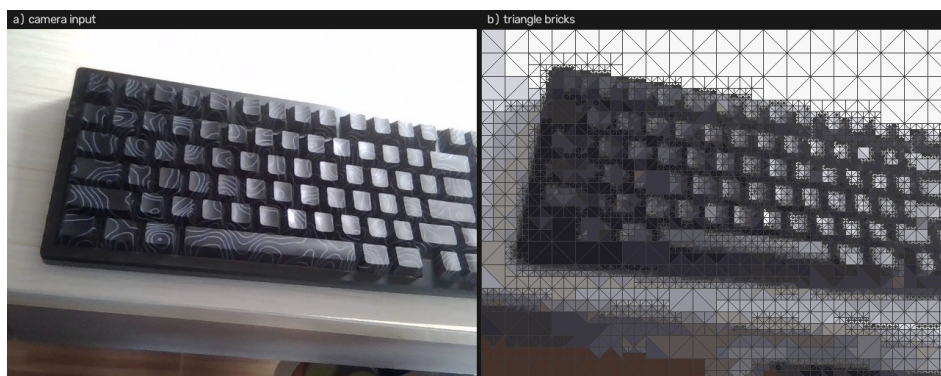


Figure 18. Mechanical keyboard. The densest high-frequency texture in the set. Two different things are being asked of the pipeline at once: the partitioner has to be small enough to resolve individual keycaps, and the palette has to keep adjacent keys from collapsing into one another. The key field is covered in the smallest cells in the whole sweep, while the desk below it — a much larger, much flatter region — stays in large cells. The keys themselves carry a marbled pattern, and the render reproduces it as mottling rather than resolving it, which is an honest limit of a 16-colour palette on a texture that varies in hue at a scale smaller than one brick.

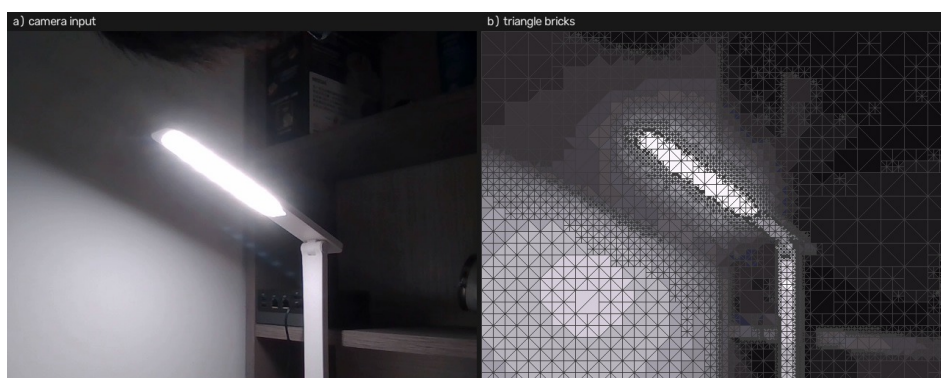


Figure 19. Desk lamp in a dim room, the extreme dynamic-range case. The light source is clipped in the input, and the render keeps it legible rather than bleeding it across the wall, because the saturated region attracts the finest cells. The surrounding shadows are the weak part: they are posterised into large flat cells, and the dark side of the frame picks up a visible cool cast from the palette's dark end. This is the case that motivated capturing with `-lock-ae` — an unlocked camera would normalise the exposure differently frame to frame and the mosaic would drift with it, so the flag is what makes a fixed-palette assumption safe here.

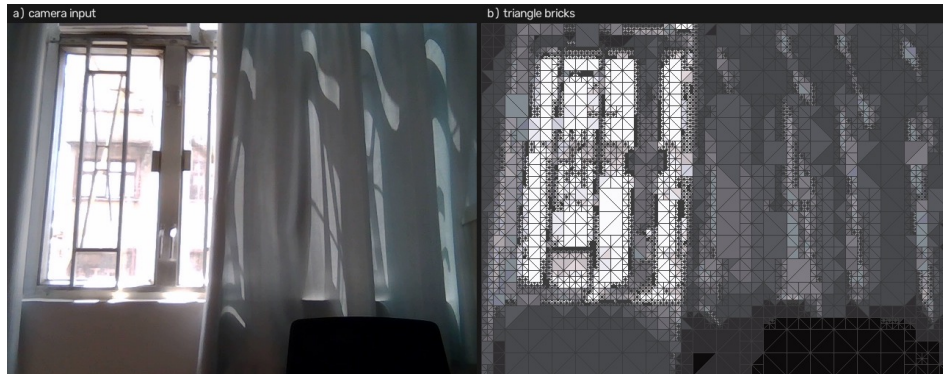


Figure 20. Sheer curtain against a window. Almost all of the structure in this frame is in the shading rather than the hue: the folds barely change colour, so what carries them is brick density — the fold boundaries get rows of small cells and the smooth panels between them get large ones. The window frame stays visible as a hard geometric boundary. It is the clearest evidence in the set that the tessellation is responding to local contrast, not to colour boundaries.



Figure 21. Dim wall beside a bright window — the weak case, kept in the set precisely because it is the one that behaves worst. The curtain, the window edge and the chair survive, so the subject is not lost. What is lost is the smooth low-contrast gradient across the large wall: the input shows a gentle falloff, and the render replaces it with a few near-flat bands with a discrete step between them, because a 16-colour palette quantises a slow gradient into visible terraces. This is the expected behaviour of a fixed palette on low-contrast content, and it is the failure I would fix first — not by raising K , but with a dithered or error-diffusion assignment, which spreads a sub-palette-step difference across neighbouring bricks instead of banding it.

6.6 Robustness in a real scenario, and summary of the measured changes

Robustness in a real scenario comes down to four distinct failure modes, and I answer each with a measurement rather than an intention. The evidence lives in the sections named.

- **Does it stay within the brick budget on unseen content?** The partitioner cannot exceed the budget by construction — the quadtree guard is `while heap and n_tri + 6 <= max_triangles`, and `region_merge_partition` raises rather than return an over-budget partition. The other hard constraint, no gaps and no overlaps, is verified rather than argued: `code/verify_coverage.py` checks all three delivered tessellations and finds cells distinct and grid-aligned, each cell’s two triangles satisfying the exact area identity $\text{area}(a) + \text{area}(b) = s^2$, and **0 uncovered pixels** in the rasterised crop (Section 1).
- **Does it survive a deadline?** Measured per-stage and paired before/after throughout Section 6.4, from 2,140 ms down to 59.5 ms per moving frame, with the spatial optimisations exact to 0 differing pixels.
- **Does it stay visually stable frame to frame?** The temporal layer and its measured churn numbers are in Section 6.4 step 8; churn on a static scene goes 0.101% $\rightarrow$ 0.000% of pixels. The conditions that make this safe — a locked exposure — are stated there rather than assumed, and I flag that the flags are opt-in.
- **Does it behave on content I did not design for?** Section 6.5 runs seven scenes spanning two portraits, a dense mechanical keyboard, saturated luggage, a clipped light source in a dim room, a backlit curtain and a large low-contrast wall, each chosen to stress a different part of the pipeline (Table 14). It reports the one case that behaves badly — the banded wall gradient — rather than only the six that work, and states what I would fix first.

Where robustness is *not* established, I would rather say so: this is a single camera on a single machine (Section 9), the test content is one image for Tasks 2–3, and the temporal layer deliberately trades a small amount of fidelity for stability.

Every optimisation I made in Section 6.4 is a paired before/after with stage ms, whole-frame FPS, ΔE /PSNR, and a pixel-diff check, so each change is documented against the measurement that motivated it. The headline reads cleanly only when each number’s measurement conditions are attached. With that in mind, and separating three different things that are easy to conflate:

- **Per-frame cost at the full quality config, moving input.** 2,140 ms before optimisation $\rightarrow$ **59.5 ms**, i.e. ~ 16.8 FPS processing-only on synthetic frames (Table 12). An intermediate build, after the two $O(T \cdot H \cdot W)$ /per-candidate fixes but before the render and means work, measured ~ 215 ms (~ 4.6 FPS); that was a waypoint, not the delivered figure.
- **The same pipeline on a static scene.** 68.9 $\rightarrow$ **11.4 ms** (~ 88 FPS) with exact spatial reuse. The 37.1 FPS in step 8 is the step-5 measurement of this (27.5 ms) and is superseded.
- **The camera’s own ceiling.** Its 31–33 FPS read-back is a separate limit and is not something these optimisations change.

The spatial optimisations are exact, with 0 differing pixels on identical input. The temporal layer (palette cadence, warm-start, dead-band, hysteresis, freeze-on-reuse) is a deliberate frame-to-frame approximation made for stability, and it does trade a small amount of fidelity for that stability, which is why paired ratios, not absolute FPS, are the trustworthy numbers here (see Section 6.2).

6.7 Problems found and solved (Task 4)

The 0.45 FPS baseline came from two main offenders: the $O(T \cdot H \cdot W)$ means stage and the per-candidate numpy means inside the partition (Section 6.2). I solved them with in-cell means, SAT tables, batched render, precomputed priorities, palette cadence, and vectorised triangles, all detailed in Section 6.4.

A second issue was fringe-contaminated means, a latent bug in the shipped reference implementation that also affected my Task 2/3 runs. The fix is in Section 6.4 step 1, the measurement is in Section 6.4, and I disclosed the residual impact on the recorded Task 2/3 numbers there as well.

K-Means palette instability was the third problem. The combination of a global RNG and 10 restarts on near-degenerate data was flipping local optima between frames (Section 6.4 step 4). I addressed it with warm-start plus dead-band plus reuse plus AE/WB lock (Section 6.4 steps 5 and 8).

Two presentation defects also needed attention, namely the HUD FPS misreport and the window stretch on resize. Their fixes are in Section 6.3 and Section 6.4 step 7.

Finally, the `build_palette` “anomaly” turned out to be a 2-sample measurement artifact, not a real performance problem, as I worked out in Section 6.2.

7 GenAI usage

I used GenAI (PolyU GenAI and a general-purpose LLM) on every GenAI-collaboration task. Across the three tasks I logged 17 prompts: 14 were adopted, and 3 were rejected or rolled back after measurement.

The division of labour was the same on all three tasks: I set the intent and the direction, and the model did the practical work. It came in three places.

Algorithm search. For each task I asked for candidate algorithms and the trade-offs between them, and I chose. This is why the geometry, the palette and the partitioner are each a deliberate choice with a recorded alternative rather than the first thing the model suggested: Otsu over a naive threshold for Task 2, K-Means-Lab over Median-Cut for Task 3, and the top-down RDO quadtree over bottom-up region merging, with region-merge implemented anyway so the comparison would be measured rather than assumed.

Code writing. The model wrote the code; I reviewed it before running it. The review is not a formality, and two of the three defects I fixed were logic errors that compiled and ran cleanly (the inverted grid scan in Section 4.3 and the inverted quadtree loop in Section 5.5). I changed both by hand after tracing the control flow myself.

Report writing. I decided the argument and wrote the outline; the model drafted the prose against that outline; I checked every number against the stored metrics and rewrote the passages that overstated what the data supported. The measurement discipline in Section 5.2 — the noise floor, the noise audit, the declared non-reproducibility — is the part I imposed, not the part the model proposed.

The role it played, and the episodes that taught me the most about how it reasons, are below; the per-task records live in `docs/progress/Task{2,3,4}.md`.

7.1 How GenAI understood the tasks

Three episodes from this work taught me the most about how the model actually reasons.

The first is a case of correct-looking code with a wrong mental model (the Task 3 quadtree loop). My guard encoded “split down to budget”, the opposite of what it should have done. The code ran without error, and only a careful review of intent (splitting increases the count) exposed it. The lesson for me was that AI-generated code which merely executes still needs a sanity check against what the code is supposed to do.

The second is the cost-diagnosis turn in Task 4. The AI correctly identified that the bottleneck was Python-to-numpy call count rather than algorithmic complexity, and its follow-up question about “exact or approximate? state the invariant” set the tone for every later optimisation, which kept each one exact by construction.

7.2 A bias the model shares with most assistants

The third episode is a bias seen most sharply in the Task 3 $S_{\max} = 64$ episode. When I asked the model to pick $S_{\max}$, it reported “64 is the clear winner” by leading with the four metrics where 64 won and demoting the single regression (ΔE_{2000}) to a parenthetical. Only after I pushed back did it break ΔE down by region and find that the shadow area was materially worse under 64. What I took from this: the model framed its answer to match the hypothesis I had already implied, and did not volunteer that objective metrics and human perception can invert. Section 8.1 returns to this as the most consequential pattern of the whole project.

8 GenAI limitations and areas for improvement

This section is my own assessment, because the limitations of the tool are not visible in the code it produced but in the places where I had to stop trusting it. Four patterns showed up repeatedly across the four tasks, and I have arranged them by what they cost me: the bias that shaped my conclusions, the unreliability of the code itself, the model’s reluctance to go and look something up, and a memory problem that grew with the size of the project.

8.1 The bias that shaped my conclusions

The first and most consequential pattern is that a single agent in a single conversation converges toward whatever the user has already said. I noticed it when I proposed that raising $S_{\max}$ would give the quadtree more room for fine splits, and the model immediately agreed, then designed an experiment that could only demonstrate my hypothesis. It never asked whether the premise was worth interrogating. The same bias reappeared when the results came in: the model reported “ $S_{\max}=64$ is the clear winner” by leading with the four metrics where 64 won and demoting the single contradicting metric, ΔE_{2000} , to a parenthetical. The contradicting analysis, that the shadow region is clearly worse under 64, was only produced after I pushed back (Section 5.4). What concerns me most is the third variant, where the model treated a metric verdict as a perceptual verdict. The four objective metrics preferred 64; my own eyes preferred 32; the model had not anticipated that these could disagree, and so it had no framework for noticing when they did.

I was not getting a wrong answer; I was getting a fluent answer from a model that had adopted my framing as its own, and in a single continuous session there was nobody to notice. The

practice that actually helped was structural rather than clever prompting: late in the project I began running the same draft through several independent review passes, each instructed to attack it from a different angle, and letting them disagree. A second reader with no memory of how the argument was constructed is much harder to seduce than the same reader continuing a conversation.

8.2 Generated code runs without being correct

The second pattern is that generated code runs without being correct. The quadtree in Section 5.2 is the clearest example: the split guard was written as “if the triangle count is under budget, return” and the loop as “split while over budget”, which is a coherent-looking encoding of the belief that splitting reduces the count. It does the opposite. The code compiled, ran, and produced a plausible-looking mosaic that quietly used 4,320 of its 9,990 bricks. Nothing raised an error; the only symptom was that my measured edge-alignment score was slightly negative. I found it by re-reading what the code was trying to do rather than what it said, and the same class of error appeared twice more, in the merge-candidate alignment and in the grid-step scan. The mitigation I adopted is the one this report has been demonstrating throughout: treat every optimisation as a claim about invariants and write an assertion that would fail loudly if the claim were wrong, rather than trusting that correct-looking code is correct.

8.3 The model will not go and look something up

The third pattern is that the model will not go and look something up unless told to, and when it does search it may not search thoroughly enough. It proposed K-Means in Lab space for the palette without mentioning that `cv2.kmeans` depends on a global RNG, so the palette is not reproducible between runs. It recommended edge-density split priority without noting that Sobel variance computed per region and computed globally differ near region boundaries. In both cases the missing information would have changed how I ran the experiment, and neither was volunteered. My own version of the problem was subtler: not a fabricated fact, but a confidently recommended algorithm whose main drawback I had to discover in the measurements myself.

8.4 A context-memory problem that grew with the project

The fourth pattern is a context-memory problem that showed up precisely because this project grew long. As the report accumulated numbers, figures and decisions, the assistant began to lose track of what had already been established, and on several occasions it produced figures that contradicted data it had itself recorded earlier, or that did not match the evidence on disk. When I audited the final draft I found mismatches of exactly this kind: values that disagreed with the numbers in the same report, image captions that described content the file did not contain, and a couple of quantities that were simply invented rather than measured. None of these came from malice and most were plausible in isolation; they came from the model reasoning over a context too long for it to hold every earlier claim and cross-check each new one against it. The consequence is a rule I now apply without exception: for a report of this size, every figure, image and code reference has to pass either an independent cross-audit or a manual re-check against the underlying files, because the model’s own confidence is not a reliable signal of whether a number is real.

What I would change for a future project is mostly about process rather than about the model. I would put the adversarial review pass in from the start rather than near the end, because it is the only mechanism I found that reliably broke the agreement. I would require that any claim

about a library behaviour, a metric definition, or a published method be backed by a reference or a run I performed myself, instead of being accepted because it sounded right. And I would treat the assistant as a fast colleague whose proposals are hypotheses to be tested, not as an authority whose output is merely to be formatted. The parts of this project that worked were the parts where I was checking: measuring before and after, asserting invariants, and being willing to reject a suggestion after the numbers disagreed with it. The parts that failed were the parts where I stopped checking.

9 Conclusion

I built a triangle-brick image-generation system that converts an image into right-isosceles-triangle mosaics under a 10,000-brick budget, and worked through all four stages.

In Task 2 I set up the tessellation and showed, with the 10-item metric suite, that the colour-quantisation method matters less than the geometry. On this luminance-dominated scene Otsu and K-Means came out close to each other, and the naive fixed threshold won the edge metrics while losing the colour metrics. The uniform grid’s negative EPI quantified how much the geometry mismatches the content.

Task 3 made the geometry adaptive (RDO quadtree) and the palette multi-colour (K-Means-Lab). A 15-run sweep plus a rate-distortion study on `sky.jpg` showed where the budget is best spent: $K = 16$, $S_{\max} = 32$ as the balanced setting, ΔMSE priority, quadtree. The $S_{\max} = 32$ -vs-64 result I present as a two-sided trade-off rather than a single winner, for two reasons: the sweep is single-image evidence, and a strong-hue scene may shift the palette-method and K rankings; and the sweep driver never seeds OpenCV’s RNG, so I measured its run-to-run ΔE2000 spread at ~ 0.4 and re-grounded the colour-side comparisons whose ΔE2000 gap was smaller than that on a deterministic palette (Section 5.4) or a seeded run (Section 5.6). Being precise about what that floor does and does not cover: each metric has its own floor, and Section 5.2 lists every comparison against the matching one. The K conclusion rests on a ΔE2000 gap of 2.34 against a 0.441 floor ($5.3\times$); the $S_{\max}$ and priority conclusions rest on structural gaps of $2.9\times$ to $34\times$ their own floors. Three comparisons do *not* clear their floors and I have said so rather than leaning on them: the colour side of the $S_{\max}$ trade-off, the $S_{\max}$ 64-vs-128 step, and the colour side of the priority comparison. The palette choice is the honest borderline — $1.5\times$ on ΔE2000 , and Median-Cut is better on all four structural metrics — and Section 5.6 sets out both sides of it instead of claiming a win.

In Task 4 I turned the pipeline into a live camera loop and dealt with the real-world problems the offline tasks never exercised: an $O(T \cdot H \cdot W)$ mean stage that took 75% of the frame, the AI-introduced quadtree bug, a latent fringe defect, frame-to-frame jitter from unstable K-Means and camera exposure drift, and a misleading HUD. I measured each before and after; on a static scene the loop reaches 11.4 ms per frame, about 88 FPS (headless, no camera read-back) through exact spatial reuse plus the declared temporal-stability layer, against 2,140 ms before any optimisation.

The pipeline’s layered design, a shared geometry/colour/rendering core with one decision layer per task, is what let the Task 4 optimisations be exact (0 differing pixels on identical input) while keeping the Task 2/3 geometry reproducible. I declare the K-Means-Lab palette as run-to-run non-reproducible, and I re-generate all headline numbers from `code/pics/task3/best/*_metrics.json` under a fixed seed.

The limitations I want to be honest about: a single test image for Task 2 and Task 3, K-Means non-reproducibility (declared, bounded), and measurements from a single machine for Task 4. The multi-scene testing (Section 6.5) covers seven frames from one camera in one room, which is

enough to show the pipeline is not tuned to one image and not enough to claim generalisation.

References

- [1] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [2] Z. Wang, E. P. Simoncelli, and A. C. Bovik, “Multiscale structural similarity for image quality assessment,” in *Proc. 37th Asilomar Conf. on Signals, Systems and Computers*, vol. 2, pp. 1398–1402, 2003.
- [3] G. Sharma, W. Wu, and E. N. Dalal, “The CIEDE2000 color-difference formula: implementation notes, supplementary test data, and mathematical observations,” *Color Research & Application*, vol. 30, no. 1, pp. 21–30, 2005.
- [4] J. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.
- [5] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [6] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [7] D. Arthur and S. Vassilvitskii, “k-means++: the advantages of careful seeding,” in *Proc. 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1027–1035, 2007.
- [8] P. Heckbert, “Color image quantization for frame buffer display,” *Computer Graphics (SIGGRAPH ’82)*, vol. 16, no. 3, pp. 297–307, 1982.
- [9] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.

A Definitions of the evaluation metrics

The image-quality numbers come from `code/brick_metrics.py`; the timing figures come from the `StageTimer` in `code/brick_io.py`, and the marginal-benefit figures from `code/task3_marginal.py`. This appendix covers the quality metrics, which are the ones whose meaning is easy to misread. Each answers a different question about the same render, so the results can be read without guessing.

PSNR (higher is better) is the peak signal-to-noise ratio between the source and the render, computed per channel over the whole image and reported in dB:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right), \quad \text{MAX} = 255$$

It is the most familiar number here and the least discriminating, because a large uniform colour shift costs it as much as a small structural error.

SSIM (higher is better) is the structural similarity index of Wang et al. [1], computed per channel on a sliding odd window of at most 7×7 pixels (smaller only if an image dimension forces it) and averaged. SSIM scores local contrast and luminance structure rather than raw

pixel agreement, so it penalises a blurred or structurally displaced render more than a uniformly brightness-shifted one.

MS-SSIM (higher is better) is the mean of the SSIM score computed at four progressively downsampled resolutions, using Gaussian-pyramid decimation (`cv2.pyrDown`) between levels. Averaging across scales rewards structure that survives decimation, so it is harder to satisfy with a good low-resolution match and noise at the finest scale. This is a four-level arithmetic mean of per-level SSIM, which is a simplification of the weighted formulation in Wang et al. [2]; it is not the standard MS-SSIM weighting, and the numbers in this report should be read as a multi-scale consistency check rather than as a comparable-to-literature score.

ΔE_{2000} (lower is better) is the mean per-pixel CIEDE2000 [3] colour difference between source and render, computed after converting both from sRGB through CIE Lab. It is the perceptual colour metric of the suite; roughly, a value above 2 is a clearly visible colour difference, and the values in this report (roughly 6 to 12) correspond to differences that are obvious on side-by-side inspection.

Edge F1, precision and recall (higher is better) compare Canny [4] edge maps of the source and the render, both extracted with $\sigma = 2.0$. Because a triangle boundary can sit a pixel away from a true edge and still look aligned, each edge map is dilated by a 3-pixel tolerance and an edge pixel counts as matched when it falls inside the other map’s dilated version:

$$\begin{aligned}\text{precision} &= \frac{|\text{render edges near a source edge}|}{|\text{all render edges}|} \\ \text{recall} &= \frac{|\text{source edges near a render edge}|}{|\text{all source edges}|} \\ \text{F1} &= \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}\end{aligned}$$

Note that the two ratios use different reference sets: precision asks how much of what the mosaic drew is real structure, recall asks how much of the real structure the mosaic reproduced. The dilation is one-sided, so a match just outside the tolerance band is missed by both directions.

EPI, edge-alignment correlation (signed, higher is better) is the Pearson correlation between the mean-centred Sobel gradient-magnitude fields of source and render:

$$\text{EPI} = \text{corr}(|\nabla \text{source}|, |\nabla \text{render}|)$$

It measures whether edge energy in the mosaic is co-located with edge energy in the source, independent of overall intensity. A positive value means the two agree about where the edges are; a negative value means they disagree, which in this project is the signature of triangle boundaries that do not follow image content. Because the magnitudes are sparse and heavy-tailed, a correct mosaic still scores near zero, so the sign is the reliable part of the metric.

That has a consequence I should be explicit about, because the rest of the report leans on EPI differences: because a good mosaic sits near zero, *magnitude comparisons between two good mosaics are not meaningful in absolute terms* — +0.045 and +0.015 are both “close to zero and positive”, and the ratio between them is not a quantity the metric defines. Where I compare EPI between configurations (Sections 5.2 and 5.6) I use it as a sign-and-ordering signal on configurations whose Edge F1 also moves in the same direction, never as a standalone effect size, and I check each such claim against the 0.0021 run-to-run spread I measured. The one comparison that rests on EPI’s *sign* rather than its size is the Sobel-priority run, which is the only configuration in the sweep that goes negative at all. The name is local to this report: it is a plain Pearson correlation of Sobel magnitudes and is not the Edge-Preservation Index of the image-fusion literature.

Quantisation error (lower is better) is the mean CIEDE2000 distance between each triangle’s own mean colour and the palette entry it was assigned. It isolates the colour-quantisation half of the pipeline from the geometry half, because it never looks at the rendered image. A high quantisation error with a reasonable PSNR means the geometry is fine but the palette cannot represent the image, which is the diagnostic that separated the K sweep from the $S_{\max}$ sweep in Section 5.2.

Budget utilisation is not a quality metric but the fraction of the 10,000 bricks the run consumed, $N/10000$. All Task 3 runs sit at 0.998 to 0.999.

Framing of the suite. The metrics are deliberately not redundant. PSNR and quantisation error see colour; SSIM and MS-SSIM see structure at one and many scales; ΔE_{2000} sees perceptual colour distance; edge F1 sees whether detected edges coincide; EPI sees whether edge energy is co-located; and the residual map in Fig. 3 shows where the two images disagree rather than how much. The disagreements between them, rather than any single score, are what produced the findings in Sections 4 and 5.4.